

MEASURE — can we believe a number

Owens: engine/mew.js , engine/sprt.js , engine/provenance.js , engine/status.js , engine/backtest_winrate.js , engine/paired_h2h.js , engine/feature_engine_contrast.js , the noise floor, the corpus stamps, and the MAG refit.

Its one number: leaf calibration — when the leaf says 90%, is it 90%.

May not: change a policy, a search knob or an engine mechanic. This division builds the rulers; it does not compete on them.

<!-- GENERATED: engine/status.js -->

MEASURE — can we believe a number

leaf calibration: live in-game leaf is WORSE than a coin on Brier (paired +0.0502, 95% CI 0.0371 to 0.0634) (n=1378 games, 200 rollouts each (2026-08-04 07:09))
when it says 90-100% it wins 54% (n=56); when it says 0-10% it wins 54% (n=52) — ECE 0.1811
powered for MDE 53.8% held-out / 51.7% full corpus; the prior effect needed n=2835
PRE-CHANGE — measured against a different build of: engine/medicham2-browser.js, engine/rollout_leaf.js
(the corpus has grown since: data/games.ladder.jsonl — more power available, not staleness)
provenance: 6 unsafe, 1 void (declared), 46 possibly stale, 55 ok, 0 missing
RATCHET TRIPPED — the unstamped list grew; provenance.js exited non-zero: diff-swarm.json, rerun-list.js
their generators ship without recording what CONTENT they read — stamp source_digests
click censoring: 1,383 of 249,404 recorded actions were NOT clicks (0.555%) and left the labeled set; 3,125
classifier vs the raw protocol on 6,205 games (67.2% of the corpus): encore recall 99.7% precision 96.7%
EM recovers 91.4% of a planted censoring bias of 0.957 against a 0.326 noise floor (amplified regime)
behaviour on the OUTPLAYED turns, after - before, paired and game-bootstrapped:
redirection turns, mass on the candidate set +0.000122 [-0.000261, 0.000514] (contains zero) n=65
coerced turns, P(the coerced action) -0.002613 [-0.003650, -0.001672] n=293 (lower is better)
CONTROL, clean turns, logL +0.000485 [0.000189, 0.000777] n=47331
refit edge: FIXTURE ONLY — not clean, and not owed either; only one of the two instruments spoke
feature_fixture --check passes: all 58 columns hash-identical to fit time — but the corpus contrast is
moved after the fit: engine/medicham2-browser.js 2026-08-06 23:33
moved after the fit: engine/board.js 2026-08-05 19:44
moved after the fit: data/engine-data.js 2026-08-06 19:20
moved after the fit: data/abra-tags.js 2026-08-06 19:19

stamped 2026-08-07 00:43

<!-- /GENERATED -->

Why the refit lives here, not in ENGINE or SEARCH

The refit is the expensive event on the one expensive edge, and it invalidates seven artifacts: counterplay, winrate-backtest, opponent-calibration, weight-multiplicity, then the mag / mew / scoreboard bundles. provenance.js derives that set rather than carrying a typed list of it.

The division that owns *knowing when a number stopped being true* is the one that should be pulling that trigger.

A restamp is only valid if the feature FUNCTION is unchanged. Damage table moved → refit. Not a restamp. There is no version of this where the shortcut is fine.

Open — in priority order

o. THE HEADLINE METRIC IS NOW EXPLOITABILITY, AND THIS DIVISION DOES NOT HAVE ONE — 2026-08-06 (3.59.0)

ADR-003 is accepted and published across the docs in this pass. **Exploitability replaces win rate as the project's headline metric, and the published comparator is VGC-Bench's approximately 100%.** That lands on this division, because the instrument is `engine/exploit.js` → `data/exploitability.json`, and that artifact is the one thing in the repository that is *declared void*.

Nothing here is a new measurement. The reframe rests on a number somebody else published and on a speed benchmark run earlier tonight. This entry records what the reframe costs MEASURE.

1. The headline metric has no value. `data/exploitability.json` is `void: true` and `provenance.js --strict` exits non-zero on it — §5g of this file has the full account. The 2026-07-26 figure was fitted on 17 features against the 58 shipped, on an engine 25 wire-fixes old, before the quality filter existed. The 2026-08-04 re-run is void because `data/policy-weights.json` — the defender — was refitted at 22:15:24 UTC while it was running. **So the comparison this project now leads with is a comparison it has set up and not made.** Say that, in those words, until it is made.

2. It raises the bar on the frozen-release discipline, and the evidence is our own. An exploitability run needs a best response trained against a *frozen* agent over thousands of games. The 2026-08-04 void **was** an exploitability run. That is a demonstrated failure mode on this exact measurement, not a hypothetical, and it is why the release boundary is a precondition rather than a nicety. `engine/exploit.js` stamps no engine digest and no digest of the target vector, which is why nothing caught it at the time.

3. The comparability argument is sound and should be stated whenever the number is. Their checkpoints are Reg M-A, ours Reg M-B, and their own paper shows policies do not transfer across team sets — a head-to-head is impossible. **Exploitability is intrinsic:** it is defined against a best response trained against *you*, in *your* format. So the two numbers sit on one scale although the two agents can never meet. This is the rare case where a comparator is legitimate without a shared population, and the reason has to travel with the figure or somebody will eventually read it as a head-to-head.

4. A noise floor for exploitability does not exist and is not obvious. §6 of this file says the floor belongs to the measurement rather than to a global constant. For a hill-climb the natural split-half does not apply — the arms are not exchangeable. The 2026-08-04 run gives the shape of the problem rather than a floor: it accepted **1 of 24** steps and its step scale decayed to 0.0168, so from round ~10 it was perturbing a near-copy of MAG. **An attack that dies in 58 dimensions returns an uninformative null on a still tree too**, which means a low exploitability figure from this tool is not yet distinguishable from the tool failing. That is the first thing to fix, and it is upstream of producing any number at all.

5. What this division must NOT do with the reframe. It must not report an exploitability number computed against a moving target, and it must not report an interim one. The SPRT rule applies with full force here: 66.7% became 44% and 57.7% became 50% in this project because somebody looked early. An adversarial search that is watched while it climbs is the same failure with a different name.

0a. ENGINE SPEED IS UNMEASURED BY THIS DIVISION'S STANDARDS, AND IT DECIDED AN ARCHITECTURE

Three readings of MEDICHAM's throughput are on record and they disagree by an order of magnitude: 3,401 battles/sec (ADR-001, July), 1,606 battles/sec (ROADMAP #61) and 13,041 turns/sec (the 3.59.0 re-measurement). The published ratio against `champions_sim` moves with them: **117x, corrected to 24.9x**. All three are now stated in ADR-001, ADR-002, `docs/MODELS.md`, the white paper, the deck, `docs/SUMMARY.md` and the technical docs, with the July figures kept and annotated rather than rewritten.

Everything wrong with this is a MEASURE problem, and none of it is an ENGINE problem.

- **No artifact holds any of the three.** There is no `data/*.json` for engine speed. A figure that decided the project's largest architectural decision has never been through the machinery every win rate in this repository goes through.
- **No generator exists.** There is no script in the repository that runs the comparison, so none of the three can be reproduced, and the two that disagree cannot be adjudicated.
- **No ratchet.** Nothing fails when engine speed regresses. ROADMAP #61 already recorded that a 2x regression went unseen for that reason; the same hole let a 4.7x error in a published ratio survive two weeks.
- **The unit was doing damage.** `turns/sec` compares the two engines and `battles/sec` does not, because MEDICHAM was driven to its 60-turn cap and Showdown with `choose('default')` to a natural end — so a "battle" is not the same quantity of work on the two sides. The 7.7x battles/sec ratio measured tonight is **not** a like-for-like number and must not be quoted as one. Two of the three historical readings are in the wrong unit for the comparison they were used to justify.

The correct fix is a stamped artifact with a declared method, not a fourth reading, and it is the same fix this division applied to the four rollout gates: a generator, a sidecar recording the machine and the configuration, and a floor. Filed here rather than done — it is a new measurement, and this pass was a publication pass.

One location is left uncorrected and is flagged rather than edited: `engine/champions_sim.js` lines 10 and 26 still state the 117x in the file header. It is ENGINE's file and ENGINE is working in the tree tonight.

1. LEAF CALIBRATION — MEASURED 2026-08-04. The leaf is not calibrated, and the claim is now powered.

`data/winrate-backtest.json` was re-derived against the current engine, on the whole clean corpus instead of a 350-game subsample, and it publishes a reliability curve. The finding is worse than the verdict string it replaces, and worse in a specific and actionable way.

The old number scored a leaf no live decision calls. It measured `winProb2` — `battle()` at MEDICHAM's default 20-turn horizon with entry effects re-fired. MILTANK calls neither: its team-preview leaf is a greedy payout at `maxTurns=60` with `seeded:true`, and its in-game leaf is `rollout_leaf.rolloutWinProb` at `explore=1.0 / foePolicy=uniform / maxTurns=60`. All three are now scored on identical positions, so the difference between them is about the leaf.

Confidence carries no information. The curve is close to a horizontal line:

leaf	says 0-10%	says 90-100%	discrimination	Brier vs coin (paired)
in-game, 200 rollouts, held-out n=1,378	wins 53.8% (n=52)	wins 53.6% (n=56)	50.99% [48.3, 53.7]	+0.0502 [0.0371, 0.0628]
preview, 40 rollouts, full clean n=6,886	wins 45.7% (n=831)	wins 55.3% (n=933)	53.22% [52.0, 54.4]	+0.0740 [0.0668, 0.0813]

Positive is worse. Both leaves are decisively **worse than a coin** on Brier and on log-loss, and worse than player-Elo, paired on the games where both have an opinion. The preview leaf puts **25.6% of all its predictions into the two extreme buckets**, where it is wrong by ~40 points.

Discrimination and calibration are separate failures needing separate answers. The preview leaf does rank — 53.22% on 6,700 decisive calls, $p < 1e-4$ — real, but only ~1.9 points above the split-half noise floor. The in-game leaf does not rank at all: 50.99%, $p = 0.47$. Randomising the playout bought variance and spent the signal.

What this is not. Nothing here says the engine is broken. The legacy `winProb2` leaf reproduces the 2026-08-02 number closely on the current engine — held-out log-loss **1.0243** against the **1.0748** published, discrimination **51.94%** against **52.63%** — so the twenty-two engine commits in between did not move the headline. Do not spend this finding on a mechanics hunt.

Also fixed here: the split was cutting on **store append order**, not date, and the store carries 4,775 date inversions. A side-symmetry witness scores 400 boards from both sides and reports $\text{mean}(p_1 + p_2 - 1) = -0.0099$, so no side advantage inside the engine is contaminating the result.

Still open, in order:

- the leaf is scored at **turn 0**, because that is where a game-outcome label exists. `rollout_r1.js` scores mid-game positions and is the other half of this; the two have never been read together.
- the **horizon** is the first suspect. `battleResult` falls back to bodies-then-HP whenever the playout does not finish, so a confident number can be a material count wearing a probability's clothes. That is a SEARCH change, not a MEASURE one — file it, do not fix it here.
- `data/winrate-backtest-rows.jsonl` holds the per-game predictions, so the curve can be re-cut without re-running the ~15 minutes of rollouts.

2. R4 has an artifact — CLOSED 2026-08-04

`engine/rollout_r4.js` writes `data/rollout-r4.json`, and `status.js` now prints the verdict out of that file instead of `NO ARTIFACT`. It does not re-pair anything: it shells out to `sprt.js --verify` and `paired_h2h.js` and refuses to write if they disagree, the same way `status.js` shells out to `provenance.js`.

The remembered 55.5% held. **ACCEPT H1 — MILTANK takes 55.5% of 535 DECISIVE PAIRS, decided after 522 of them, LLR 3.00 against a 2.94 bound.** The corpus is 5,248 lines, which is 2,624 games, which is 1,312 seed pairs — the store writes a log-only companion record under the same id, so a line count double-counts every game and the handoff's "5,248 games" was exactly twice the truth. The artifact records all four numbers and asserts the invariant that makes them relate.

Two things it is not. The point estimate is **stopped at a boundary**, so it is biased high and the 95% CI beside it is a fixed-n formula quoted for context, not the inference — read the verdict. And `status.js` classes the corpus **PRE-CHANGE**: `engine/medicham2-browser.js` moved 04:47, the games were played 04:41. Both arms shared the pre-fix rollout model, so the contrast is fair and the run stands as a measurement *of that build*; that the edge survives into HEAD is an assumption. It gets re-run at the next frozen engine release.

No A/A run exists for this comparison, so the noise floor is **not established**. The substitute in the artifact is three independent split-half cuts of this run: spreads of 0.2, 3.9 and 1.3 points against an effect of 5.5. One cut alone would have been useless — the spread of a single split-half is itself a draw with sd about 4.3 points at this sample size.

3. R1 has an artifact, and it does not say what the docs said — CLOSED 2026-08-04

`engine/rollout_r1_artifact.js` writes `data/rollout-r1.json` from the committed row dump, and `status.js` prints its verdict. The gate previously read a file of the same name that `engine/rollout_r1_join.py` wrote for the **withdrawn** cross-language join — nothing was hidden, the join prints its own withdrawal, but the gate

read it because it owned the filename. The join is now `data/rollout-r1-withdrawn-join.json` with `withdrawn: true`, and `status.js` refuses to print any artifact carrying that field.

The recomputation does not reproduce the published PASS. `docs/ROLLOUT-design.md` claimed 68.18% against material's 65.26%, +2.91 [1.79, 4.04]. From `data/rollout-r1-rows.jsonl` the same formulas give **65.72% against 65.26%, +0.46, 95% CI [-0.72, +1.63] — UNDECIDED** on 9,201 positions.

The material column matches the published figure to the digit, so it is the same sample. The rollout column reproduces §4.2.1's **greedy** calibration table bin-for-bin, so the surviving dump is the `explore=0` incumbent and the `explore=1` run that produced 68.18% left no file. That is the lesson, not the arithmetic: **the dump stamped no N, no explore and no build digest, so two runs four accuracy points apart were byte-indistinguishable.** `rollout_r1.js` now writes `data/rollout-r1-rows.meta.json` beside every dump. R2 and R3 still have the same hole.

The split-half spread of this run ranges 0.43 to 2.01 points against an effect of 0.46 — the effect is inside its own noise floor, which is an independent route to the same UNDECIDED.

Open consequence, filed to SEARCH: `--rollout-explore` defaults to 1.0 and `engine/rollout_leaf.js:147`, `engine/mag_bot.js:145` and `docs/MILTANK.md` all cite 68.18% as the reason. Re-running `EXPLORE_LIST=1 DUMP=rollout-r1-rows.jsonl node engine/rollout_r1.js` and then `node engine/rollout_r1_artifact.js` settles it, and R2 says the leaf is cheap.

SEARCH RAN IT, 2026-08-04. The published figure reproduces. `data/rollout-r1-explore-sweep.json` and `data/rollout-r1-explore1.json`: on the identical 9,201 positions, `explore=1.0` judges at **67.971%** against the published 68.18%, and its lift over the same material baseline is **+2.706 [1.596, 3.817]** against the published +2.91. Paired against the greedy arm it is **+2.25, 95% CI [1.31, 3.19]**, monotone in `explore` ($0 \rightarrow 0.5 \rightarrow 1.0 = 65.72 \rightarrow 67.58 \rightarrow 67.97$) and it holds at the live 60-turn horizon. **The retraction was right about the provenance and wrong as a guide to the arm — R1 is UNDECIDED on the incumbent and a PASS on the arm that ships. Three things this division should act on:** 1. **The command in this section would have destroyed the evidence.** `DUMP` resolves under `data/`, so `DUMP=rollout-r1-rows.jsonl` overwrites the committed greedy dump — the only record of the incumbent arm, committed for exactly that reason. SEARCH used a new filename. Worse, the sidecar path in `rollout_r1.js:338` is the hardcoded literal `data/rollout-r1-rows.meta.json` whatever `DUMP` is set to, so it lands beside the wrong dump. `rollout_r1_artifact.js` rejects it on the name-and-row-count check, which is the check working — but the fix is to derive the sidecar path from `DUMP`. 2. `status.js:229` **still prints the greedy arm as "R1 leaf accuracy".** SEARCH did not overwrite `data/rollout-r1.json`, deliberately: it is this division's artifact, written hours earlier, and it is the only record of the incumbent. But the line now reads UNDECIDED for a configuration the bot does not run. One line — point it at `rollout-r1-explore-sweep.json`, or print both arms. 3. `engine/mew.js` **exposes no** `--miltank-explore`. So the question this settles — which playout JUDGES better — cannot be escalated to the one that matters, which playout WINS more. R4 was itself run at `explore=1.0` and cannot arbitrate its own setting. Two parsed flags on `mew.js` and the A/B becomes runnable. One hypothesis this division filed to SEARCH is **measured and rejected:** `battleResult` scoring bodies-then-HP on unfinished playouts is real but is not the mechanism. Over 1.1M playouts, 99.5–99.8% end by an actual wipeout at every `explore` setting and at horizons 20 and 60; cap-hits are 0.2–0.5%. Exploration makes playouts longer (4.4 → 6.1 mean turns), not truncated. Filed to ENGINE as a latent hazard. The flat reliability curve in `data/winrate-backtest.json` needs another explanation — and note that on human corpus positions the `explore=1.0` leaf is **not** flat: its ECE is 0.104 with a monotone curve running 0.166 → 0.842, against 0.196 for greedy.

4. R2 and R3 stamp their configuration now — and R3's published number has no control

`engine/run_stamp.js` is one implementation of the sidecar `rollout_r1.js` hand-rolled inline, so the next gate cannot grow a second format. It writes `<artifact>.meta.json` — the same convention that makes `data/rollout-r1-rows.meta.json` describe `data/rollout-r1-rows.jsonl` — carrying N, explore, every knob including the ones left at a default, sha256 content digests of every source the gate reaches, the commit, and **whether the tree was dirty**. A clean commit id over a dirty tree is a lie of exactly the kind this exists to stop.

Both published numbers reproduce as arithmetic, and neither reproduction is worth much. That is the finding, and it is a different finding from R1's.

gate	published	recomputed from committed evidence	reproduces
R2	477 boards over 200 games; 5.83 ms median at n=10	the affordability table (K=3 → 0.47 s median, 1.75 s worst; K=4 → 1.49 s / 5.53 s) reproduces to the digit from <code>leafCostMs</code>	derived layer yes, base layer NOT CHECKABLE
R3	72.9% over 70 decisions (19 agreed, 20 skipped)	$100 \times (70 - 19) / 70 = 72.857142857142854$, bit-identical to the stored float	yes, and it is a tautology

R3's divergence is a pure function of two fields in the same file. There are no per-decision rows, so "it reproduces" means the artifact is internally consistent — nothing more. R2 dumps no per-leaf timing at all, and a duration cannot be recomputed by anyone in principle: it is a fact about a machine under a load, and nothing records the CPU, the node version or what else was running. **R2 is the one rung that is re-run or it is nothing.**

THE R3 RESULT IS NOT INTERPRETABLE AS PUBLISHED, and this outranks the sidecar work. `rollout_r3.js` computes the only control that makes a divergence rate mean anything — the same search on a different seed disagreeing with **itself**, where the truth is 0.00 by construction — and it `console.log`s it and does not write it. Its own verdict branches on that number: `rate <= floor` prints NOT A RESULT. So `data/rollout-r3.json` cannot say which branch its own run took.

`docs/ROLLOUT-design.md` §5 does publish floors — 71.7 / 50.0 / 45.5 / 43.8% — but **for four earlier runs, none of them this one**. At N=20 that floor measured *higher* than the divergence. The committed artifact is a fifth run at N=600 on 70 decisions, and its floor was printed to a terminal and lost. `engine/status.js` and `docs/MILTANK.md` both quote its 72.9%, and `MILTANK.md` spends it on a decision: "so it does diverge, and the equilibrium version is worth building."

Read plainly: **the divergence is probably real** — the doc's floor fell from 71.7% to 43.8% as N rose from 20 to 200, and this run used N=600, so its floor should be lower still. But *probably* is an inference from a different run, and the Wilson interval on 51/70 is **[61.5%, 81.9%]**, which is wide enough that a 44%-class floor is the only thing separating a result from an artefact of the argmax. The next run writes the floor; until one does, the 72.9% is a headline with its control missing.

A second defect, found on the way: `data/rollout-r3.json` 's own caveat is false about the run it describes. It reads "Switch candidates are excluded and counted". Commit `b4ec80b` deleted the `if (ca.switchTo || cb.switchTo) continue;` line — switches went **on** the menu, which is what that commit was *for* — and left the string alone. It has shipped that way since 2026-08-03, and the `withSwitch` / `choseSwitch` counters that commit added were printed and never written, so its own headline ("4 of 12 when one is on the menu") lives in a commit message.

R2 timed a leaf the bot does not run. `rollout_r2.js` called `RL.rolloutWinProb` without `explore` or `maxTurns`, inheriting `engine/rollout_leaf.js:197` 's `explore = 0` and `engine/medicham2-browser.js:1079` 's `maxTurns = 20`. MILTANK's in-game leaf is **explore=1.0 at maxTurns=60** — a randomised payout at

three times the horizon. That is R1's hole in cost form: two library defaults, written down nowhere, deciding the number. Both are now explicit, overridable and stamped, with defaults that preserve the old behaviour exactly so nothing re-dates the committed artifact by accident.

Also corrected in the generators, all of them visible in `status.js` :

- `games` was the `GAMES` environment **cap**, not a count. `status.js` printed "477 boards over 200 games", so an environment variable was being read as a measurement. It is now the distinct games actually traversed, with the cap beside it as `games_requested`.
- `leafCostMs` quantiles per N were computed over possibly-different board sets — a leaf returning null at one N and not another silently misaligns the columns — and only the n=10 count was recorded. `samples_per_n` now records all of them.
- R3's disagreement-gap median was computed twice, once to print and once to store. One variable now.
- `docs/ROLLOUT-design.md` §5's "roughly 200x the simulated turns per millisecond" is **155x** by the arithmetic of the two artifacts it cites (10×20 turns / 5.83 ms against 1 / 4.52 ms), and 155x is itself a ceiling because it assumes no playout ends early. `rollout_r2.js` now prints the division instead of a remembered figure. **The doc still says 200x** — see filed, below.

Two retrospective sidecars were written by `node engine/run_stamp.js --reconstruct`, which infers the build from the commit that carried the artifact and marks every field `reconstructed: true`. Both score HIGH: `data/rollout-cost.json` was written 25 s before `05248f2`, `data/rollout-r3.json` 159 s before `b4ec80b`. That is evidence about a commit, not a record of a run, and it says so on every line — a stamp that hashed today's sources would describe the file rather than the run, which is `data/rollout-r1.json`'s own stated reason for recording null.

Filed, not fixed:

- `data/rollout-cost.json` **should be** `data/rollout-r2.json`. It is the only rung whose file does not carry its gate's name. Four readers: `engine/status.js:230`, `web/build-status.js:200` and `:265`, and the generated `web/status-data.js`. Three of the four are under `web/`, which MEASURE does not own. A rename that misses a reader prints NOT DERIVED and reads as "nobody ran this", which is worse than the inconsistency. Needs WEB in the same pass.
- `~~ n / n_unit` **on R1 and R4.~~ DONE 2026-08-04** — see §7 below.
- `~~ engine/rollout_r1_join.py` **writes a naked** `isoformat()` **~~ DONE 2026-08-04, and it was five files, not one** — see §8 below.
- `docs/ROLLOUT-design.md` **§5's 200x, and §R3's PASS**. Both are SEARCH's document and a SEARCH explore sweep is live. §5 should read 155x-at-most, and the R3 PASS should name which run it is quoting, because the floors in its table belong to runs that are not the committed artifact.
- `docs/MILTANK.md:70` **spends R3's 72.9% on a build decision** without its control. Same owner, same reason.
- `engine/rollout_r1.js` **should call** `engine/run_stamp.js` instead of its inline copy. SEARCH holds that file for the explore sweep. The shapes are identical today; two copies is how they stop being identical.

5. The possibly-stale artifacts, and the one class the checker could not see

`node engine/provenance.js` lists them; `node engine/provenance.js --graph` now prints the derived artifact graph itself, which is the part of that tool that could be silently wrong. Most entries are ordering artefacts inside a single run and are already annotated as such. The ones to actually chase are those older than `policy-weights.json`, those recording no game count at all, and — new — those carrying a **CORPUS DRIFT** note.

THE CANONICAL READER WAS HIDING ARTIFACTS FROM THE CHECKER. `provenance.js` derived an artifact's inputs by looking for a filename beside a read verb. A generator that loads the store the *recommended* way — `loadGames()` / `load_games()`, which resolve the path inside `engine/quality.js` and `engine/store.py` — never names `games.ladder.jsonl`, so it recorded **no dependency on the store at all** and was reported `ok` forever. Doing the right thing was the thing that made you invisible. Store derivation is now detected by the `LOADER CALL` (or an import of the reader), which is what a generator actually does.

Three more attribution faults surfaced with it, each of which had the same effect of exempting real artifacts from every corpus check:

- **A read** `open()` **looked like a write.** `pokemon-roles.json`, `role-matchups.json` and `roles-eval.json` were credited to `engine/build_roles_js.py`, which `READS` them to build a browser bundle, instead of `engine/roles.py`, which computes them from the store. `build_roles_js.py` touches no games, so all three were classed not-store-derived. A write test at line scope now requires a mode string.
- **The Python** `OUT = os.path.join(...)` **idiom hid a writer entirely.** `data/guru-matchups.json` — the source file at the centre of the `guru.js` divergence — had **no detected generator and was absent from the audit**. One level of variable indirection is now resolved.
- **Following into** `engine/quality.js` **classified everything as open-sheet.** `quality.js` names every store by construction, in its comments and in the error message that tells a caller how to pick one. `data/winrate-backtest.json`'s 6,886 **ladder** games were being judged against the 8,173-game open-sheet ceiling. It is a named exception now, with that reason.

The graph went from 76 artifacts (49 store-derived) to **84 artifacts (57 store-derived)**. One of the eight newly visible files, `data/counters.json`, was **older than the quality filter** — the `UNSAFE` condition this tool exists to catch, invisible for nine days. Regenerated (15 s); `provenance.js --strict` is green.

5g. AND IT WAS HIDING SEVEN MORE — the write detection had the same hole in the other language

84 artifacts → 91, 57 store-derived → 60, and two of the seven were UNSAFE. Found 2026-08-04 while writing the missing `docs/MODELS.md` entries: the roster guard reported `data/move-priors.json` as generated by `engine/state_encoder.py`, which only **reads** it. Three defects, each the same shape as §5's and each found by chasing the previous one.

- `const` **broke the path-indirection arm.** §5 taught this file the Python idiom `OUT = os.path.join(...)` ... `json.dump(..., open(OUT, "w"))`. The JavaScript spelling is `const OUT = process.argv[3] || path.join(...)`, and the capture took the `KEYWORD` and then failed on the `=`. Every generator using it scored zero. The cost is the §5 cost exactly: `engine/policy.js` loads the store through `quality.js`, `state_encoder.py` opens no game file, so **the behaviour clone that nine files read was classed not-store-derived and exempt from every corpus check here**. It is 2026-07-31 vintage — the 5,269-game era — and nothing could say so.
- **A READ assignment is not a writer, and accepting** `const` **proved it immediately.** `const r = JSON.parse(fs.readFileSync(...'regulations.json'...))` followed later by an unrelated `fs.writeFileSync(file, r.body)` credited `engine/fetch_smogon_stats.js` with generating the format registry — a one-letter identifier matching `\br\b` inside any later write. An assignment whose own right-hand side is a read verb now never establishes a writer.
- `named()` **was a substring test, and the comment claiming that was safe was FALSE for the most-read file in the repository.** `ladder.json` is a substring of `games.ladder.jsonl`, so every generator that opens the game store was recorded as naming `data/ladder.json` — which is how `engine/refresh-site-data.NOARCH.py` was credited with generating **MACHAMP's hill-climb artifact**, whose real writer is `engine/ladder.js` (the on-disk keys are `ladder.js`'s to the letter). The same fault

hung a **phantom** `ladder.json` **input** on every store reader, and `roles.js` inside `pokemon-roles.json` did it again across eight more artifacts. Those show up as "*older than its input*" notes about dependencies that do not exist. An occurrence now only counts when the name is not the prefix of a longer one.

Corrected attributions, each verified against the generator rather than trusted: `move-priors.json` → `engine/policy.js`, `ladder.json` → `engine/ladder.js`, `dynamics.json` → `engine/dynamics.js`, `rollout-r4.json` → `engine/rollout_r4.js`, `smogon-priors.json` → `engine/smogon_priors.js`. Newly visible: `bring-bias.json`, `bring-priors.json`, `brood.json`, `core-matchups.json`, `exploitability.json`, `playstyle-matchups.json`, `smogon-priors-bo3.json`.

Two of the seven were UNSAFE, and the split between them is the point.

- `data/bring-priors.json` **was genuinely UNSAFE** — five minutes older than the quality filter, so computed under a different definition of which games count. It reads the store through `quality.js`. Regenerated (30 s), and it moved a figure a long way: `n_sides` **5,368** → **14,456**, and the format's **mega rate had been measured on 62 sides and is now measured on 12,442**, `p_side_megas` **0.9355** → **0.8785**, `p_mega_is_lead` **0.5345** → **0.5159**. `CLAUDE.md` sets a domain RATE floor there — "*a game without a mega should be rare*" — and the floor was being checked against 62 sides.
- `data/exploitability.json` **is a FALSE POSITIVE of the filter rule and a TRUE negative anyway, and it is left RED**. `engine/exploit.js` reads no game store at all — it plays self-play games from `policy-weights.json` — so the quality filter has no bearing on it, and the honest fix is a `not_store_derived` declaration, which only a re-run can write. **I did not add one, deliberately**. Stamping it would make a **genuinely unquotable** artifact look clean: it is PRIORITIES #18, WOBBUFFET's 63.2% fitted on **17 features against the 53 we ship**, and it is rendered on `web/stadium.html` and `app/stadium.html` today. Re-running it is a ~4,000-game adversarial search against a mid-flight MAG, which the engine release boundary forbids.

So `node engine/provenance.js --strict` **exits 1 on one artifact, and** `tests/run-all.js` **gates on it. That is stated, not filed.** The artifact has been invalid since 2026-07-26; the only thing that changed today is that something can see it. It needs Will's call between re-running WOBBUFFET after the release boundary and pulling the number off the two stadium pages.

5a. CORPUS DRIFT — and the answer to "two definitions of clean games"

There are not two definitions. There is one, and the other number is four days old.

`data/live.js` and `data/winrate-backtest.json` said 6,943; `data/meta-usage.json`, `data/roles-eval.json` and `data/guru-matchups.json` said ~5,269. Measured rather than argued:

figure	written	what it is
6,943	2026-08-04 03:09	<code>load_games(clean=True)</code> over the store as it stood then
6,890	2026-08-04 02:52	the same predicate, 17 minutes earlier — <code>backtest_winrate.js</code> began its run then and the collector appended exactly 53 clean games while it ran
6,886	—	6,890 minus 4 games whose <code>winner</code> matches neither player's name. A genuinely narrower question, and it is already NAMED: <code>scorable</code> / <code>dropped_no_label</code>
5,269	2026-07-31 16:42	the same predicate over a store holding 29,117 collected instead of 38,587
5,265	2026-07-31 16:43	5,269 minus the same 4 unlabelable games

Collected grew ×1.325 and clean grew ×1.318 over those four days. A changed predicate does not scale with the corpus; a snapshot does. And `tests/test-quality.js` run tonight has the JS and Python readers selecting the **identical** 6,943 ids, sha `60aab8e1978e7554` on both sides. So renaming anything would have been wrong: **the**

defect was a date, not a word.

Why nothing caught it: mtime cannot. The store is append-only and its mtime moves every hour, so an mtime rule would mark every store-derived artifact stale within an hour of being rebuilt — a gate that cries wolf.

`provenance.js` now compares the **declared count** against the clean corpus and warns past 10%, which the measured growth rate (~7%/day) makes "roughly a day and a half behind". Thirteen artifacts are flagged, including all three named above at 24.1–24.2%.

Two supporting fixes, both of which were the reason the headline artifact escaped:

- `declaredGames` now reads an explicit corpus claim first — `provenance.funnel.clean`, `provenance.usable`, `corpus.clean_games`. `data/meta-usage.json` states its population more carefully than any other file in the repository and had **no key the checker looked at**, so the file that started this question was the one it could not see a count for.
- The drift check ignores a bare `games` key, and that is deliberate: `rollout_r2.js` published `games` as the GAMES environment **cap**. Until every writer says whether `games` is a corpus or a sample, a drift figure computed on it is a guess, and the fix belongs in the generator.

Do not touch `data/quality-filter.json` **to record the new funnel**. Its mtime is `FILTER_MT`; bumping it marks every older artifact UNSAFE and turns `--strict` red across the repository.

5b. `data/guru.js` said 0 where `data/guru-matchups.json` said 6 — and both were misleading

`build/build_guru_js.js` read `g.decisive`. `engine/guru.py` writes the list as `decisive_matchups`. A missing key gave `[]`, the generator then recomputed `n_decisive` from **its own empty fallback**, and shipped a provenance note asserting "ZERO statistically-decisive matchups on this population" as though it were a finding. The 144-cell matrix was byte-identical throughout, which is why nobody noticed. `venusaurmega` / `venusaur-mega`, in a new pair of files.

The true value is 6 directed = 3 distinct matchups, and the generator carries the source's count now instead of recomputing it. Three things stop it recurring, all derived: every source key must be projected or named in `DELIBERATELY_UNUSED` with a reason; the source must agree with itself (`decisive_matchups.length === min(n_decisive, 20)`); and `build_guru_js.js --check` rebuilds the bundle in memory and diffs it, run by `tests/test-guru-derived.js` on every suite run.

And the measurement underneath it: 3 of 66 pairs is what chance produces. Each cell is its own 95% test. Over 66 unordered pairs the expected number clearing that bar with no real effect is **3.3**, and **3** clear it. The smallest exact two-sided binomial p-value in the matrix is **6.1e-3** against a Benjamini-Hochberg threshold of **7.6e-4**, so **zero survive FDR at q=0.05** and zero survive Bonferroni. The bundle publishes both counts (`n_decisive`, `n_decisive_corrected`) plus the arithmetic. The old file's "ZERO decisive" string was accidentally right and arrived there by a bug — which is worse than being wrong, because it cannot be checked.

`web/index.html:1845` gates a panel headed "*These are the matchups we can actually trust*" on `GURU.decisive.length`, so it will now render three matchups that do not survive multiplicity. It should read `decisive_corrected`. WEB's file; flagged, not edited. Note the same issue already affects `isSig()` in the matrix and the "statistically significant loop" claim, independently of this fix.

`data/guru-matchups.json` is itself 24.2% behind the corpus. Regenerating it is a separate, deliberate refresh — it moves every number the GURU booth renders — and is not done here.

5c. The thirteen drifting artifacts — TRIAGED, and NONE of them is a silent refresh

`node engine/provenance.js` flags thirteen artifacts 10.6–47.2% behind the clean corpus. The question that decides what to do with each is *does regenerating it move a published figure*, and it was measured rather than guessed: every scalar in each artifact was matched against the living docs and the site pages, at headline

depth, with the universal constants (0.693, 0.25, 50%) excluded because they appear for reasons that have nothing to do with the artifact.

The answer is that there are zero safe silent refreshes in the set. Nine carry a verdict string or an interval-based claim that regenerating could flip; the other four have headline figures typed into `MODELS.md`, the white paper or `SUMMARY.md`, which `engine/sanity_check.py` §5 cross-checks. By this project's own living-docs rule, regenerating any of them is a docs pass, not a refresh.

artifact	behind	what regenerating moves	act
<code>war.json</code>	47.2%	verdict <i>"WORSE THAN A COIN AT EVERY REGULARISATION STRENGTH TESTED"</i> ; <code>held_out.log_loss</code> 0.694 in <code>MODELS</code> + white paper	STOP — a null on a corpus that has since doubled is the most interesting one here
<code>policy-eval.json</code>	43.8%	verdict <i>"phase-conditioning did not help; species-only prior retained"</i>	STOP
<code>pory-eval.json</code>	33.4%	<code>log_loss.pory</code> 0.6298 in white paper + <code>SUMMARY</code> , gated by <code>sanity_check</code>	STOP — restamped instead, see §5d
<code>pory-nn.json</code>	29.4%	Blast radius OVERSTATED in this row and corrected 2026-08-04. <code>val_logloss</code> and <code>auc</code> are not keys in the file — it holds an <code>arms</code> array with per-arm <code>logloss / acc / auc</code> . And the 71.6% in <code>MODELS.md</code> and the white paper is the policy clone's top-3 accuracy , a different measurement that happens to match. No living doc cites PORY-NN , so regenerating moves zero published figures. Regenerated	done
<code>xatu-belief.json</code>	29.3%	<code>n_games</code> 4,910 and <code>top1_accuracy.belief</code> 31.2% in <code>MODELS</code> ; an improvement CI clear of zero	STOP
<code>guru-matchups.json</code>	24.2%	every number the GURU booth renders; <code>log_loss_matchup_prior</code> 0.712 in the white paper	STOP — and explicitly not in this pass, WEB is in that booth
<code>roles-eval.json</code>	24.1%	headline <i>"0.6935 vs a coin 0.6931 and rating 0.6967"</i> — a knife-edge that regeneration can flip either way; six figures in <code>MODELS</code>	STOP
<code>pokemon-roles.json</code> , <code>role-matchups.json</code>	24.1%	same generator as <code>roles-eval</code> (<code>engine/roles.py</code>); all three move together or not at all	STOP
<code>vocab-usage.json</code>	24.1%	<code>role_coverage_of_battle_usage</code> 97.2% in <code>MODELS</code>	STOP (one-line docs pass)
<code>xatu-context.json</code>	24.1%	improvement CI [0.022, 0.042] rendered on the site	STOP
<code>meta-usage.json</code>	24.1%	nothing typed — the closest thing to a clean refresh, and PRIORITIES #16 names <code>node engine/analyze.js data/games.ladder.jsonl</code> as its closing command	ASK — <code>engine/mag_bot.js</code> and <code>engine/mew.js</code> read it, so it is the live bot's meta prior, and moving that is not MEASURE's call with an engine release boundary pending. It is not a refit trigger: <code>engine/feature_fixture.js</code> excludes it by name and <code>board.js</code> never reads it
<code>counterplay.json</code>	10.6%	<code>result.mean_coverage_gap</code> 0.0321, CI [0.0086, 0.0563] — an interval that currently excludes zero	STOP

A false positive in the drift check itself, measured not argued. `pory-eval.json` is reported 33.4% behind, and it cannot be less than ~21% behind however often it is regenerated. Its population is not *clean ladder games*; it is *clean ladder games whose raw log is present and names a winner*, a strict subset. Running the generator over the whole current corpus reaches **5,456 games, not 6,943**, so its true drift is 15.3%. Every artifact reading `games.ladder.raw-logs.jsonl` has this. `provenance.js` 's existing escape hatches (`gate` , `games_requested` , `sampled`) do not cover it, because this is neither a gate nor a deliberate sample — the artifact needs to declare the ceiling its population can reach, in the same style. Not fixed here: `provenance.js` was built tonight and a second hand on its drift arithmetic is how two files come to disagree about one fact.

5d. PORY — the artifact restamped, and the coefficients were wrong for ten days

The verdict was not stale. The generator was answering the wrong question, correctly, every run. `data/pory-eval.json` still read *"a real, calibrated value net"* ten days after PORY was retracted, and `engine/pory.py` 's gate was `hi < coin` and `hi < material_heuristic` — which is TRUE on this sample (`hi` 0.6456 against 0.6931 and 0.6550). Restamping the file alone would have been undone by the next run. `material_heuristic` is a crude 0.75/0.25/0.5 **sign** rule; beating it is arithmetic.

The tie is now measured, not inferred. Against a logistic on `[alive_diff, hp_diff]` alone — same gradient descent, same standardisation, same temporal split — PORY scores **0.629799 to 0.629778**: paired difference **+0.000021 (POR Y worse)**, 95% CI **[−0.000013, +0.000056]** clustered by game over 925 held-out games. On the current corpus (5,456 games) it is **−0.000001**, CI **[−0.000031, +0.000030]**. The retraction is robust to the corpus growth.

The reduction is structural, so no amount of data changes it. Every state is emitted from both perspectives with the label flipped, so the gradient on any column identical across the two rows cancels exactly: intercept and `turn/10` are pinned to `0.000000000`, not shrunk to it. `my_alive` and `foe_alive` swap and come back exactly antisymmetric (sum `0.000000000`). Five features, two degrees of freedom.

`engine/pory.py` **reproduces its own artifact bit-for-bit** — replayed on the identical first-4,623 clean-game sample it returned this file's weights, `feat_std` and log-loss exactly. So the fault was never the arithmetic. The gate now reads the paired difference, the withdrawn string travels under `withdrawn_verdict`, and `reduced_form` is derived from the file's own weights.

REGENERATED 2026-08-05 on the current corpus, deliberately (the dispatch Will approved). `data/pory-eval.json` now describes **5,883 games / 97,732 board-states** (was 4,623 / a 925-game test split) and **declares** `population_ceiling: 5883` — the §5f hatch, written by the generator on its first deliberate run since the hatch existed. Everything below survives the growth: paired difference vs the two-feature logistic **+0.000001**, 95% CI **[−0.000026, +0.000029]** clustered over 1,177 held-out games; the verdict string is unchanged. The numbers a document would quote moved: log-loss **0.6298** → **0.6236** [0.607, 0.6387], sign-rule heuristic **0.655** → **0.6428**, two-feature baseline **0.629778** → **0.623623**, reduced form **0.9943 / 1.4080** → **0.9809 / 1.4093**, accuracy 0.6264 → 0.630, ECE 0.017 → 0.0138. Doc locations quoting the old figures are listed in §13b; propagation is the router's pass, not this file's.

The documented coefficients had no artifact behind them — the P1 class. `1.256 / 1.544` in `docs/MODELS.md` and `web/stadium.html:342` is commit `44e0fb0` (2026-07-24, `n_games` 7,381), the last run fitted on the **unfiltered store with bot games in it**. `7f74236` put every model behind the clean filter on 2026-07-26 and the coefficients moved to 1.0259 / 1.4347, then 0.9946, 0.9962, **0.9943 / 1.4080**. The retraction has been citing bot-contaminated coefficients as its evidence ever since. `MODELS.md` is corrected with the history; `web/stadium.html:342` **still says 1.256 — WEB's file, flagged not edited**.

5e. tests/test-site-data-fresh.js — two rules in it were wrong

It kept a second definition of stale, and it was the one provenance.js **had already rejected.** The verdict-input check compared each artifact's mtime against the newest games.*.jsonl and failed past a day. The store is append-only and the collector runs hourly, so that clock cannot be beaten: five artifacts were red and **four of them are clean** by the canonical rule. It delegates to provenance.js now, the same way status.js does. The founding case survives the change — chomp-ev.json four days behind is ~28% drift, well past the 10% threshold.

What delegation loses is stated rather than dropped: drift can only see an artifact that declares a corpus, and chomp-ev.json, eval-report.json, policy-weights.json, policy-weights-joint.json and damage-validation.json declare none. They are **listed every run without failing**, so the pressure is on the generator to record a count — the shape tests/test-timestamps.js already uses.

--fix **would have refitted two models to make a freshness check go green.** The guard that stops it detected a publisher by the filename suffix -eval.json, which is not a property of anything. It caught engine/pory.py. It did **not** catch engine/nmf_roles.py (writes nmf-roles.json) or engine/xatu.py (writes xatu.json) — both fitted models quoted in MODELS.md, both on the auto-run list. The rule now is that a bundle writes only browser files and a generator that also writes a data/*.json is a publisher; checked against all ten generators this test names.

Seven of the ten stale bundles were regenerated. **Four were byte-identical apart from a date stamp** (mew.js, move-effects.js, mega-formes.js, status.js) and two entirely so (abra-meta.js, roles.js) — pure mtime, the check crying wolf. **Two had really rotted:** mag.js was serving standard errors from before the last weight change (0.02452 against policy-weights.json's 0.02363) and scoreboard.js was rendering superseded weights (1.1887 against 1.0884). That is the class this check exists for and it was real.

Also found: build_mag_data.js and build_scoreboard.js **crash without** SHOWDOWN_PATH, so --fix fails on them in any shell that has not exported it, and the test does not say so. Same shape as PO #40 — two ratchets that crashed rather than failed for the same reason.

Still red, not filed: data/pory-nn.json at **29.4% corpus drift.** The command is python engine/pory_nn.py; it is a neural-net train and it republishes val_logloss 0.612 and auc 71.6%, which MODELS.md, the white paper and SUMMARY.md all quote. That is a stop-and-ask, not a refresh.

5f. IS THE DRIFT THRESHOLD A TREADMILL? Yes, and the unit is wrong — DECIDED 2026-08-04

data/pory-nn.json was regenerated on the current corpus and tests/test-site-data-fresh.js immediately reported **CORPUS DRIFT 15.7% — declares 6,008, 7,123 clean now.** The store grew during the retrain. That is not a bug in either tool; it is what a percentage of an unbounded append-only corpus does.

The verdict: a percentage is the wrong unit, and it is not a fraction at all — it is an age. The collector runs hourly and clean games grew 5,269 → 7,123 in four days. For an artifact of age Δt , drift is $1 - n(t_0)/n(t)$, which depends only on elapsed time. A 10% threshold is therefore "about a day and a half old", stated in a unit that hides the fact that it is a clock — which is exactly what §5e removed from test-site-data-fresh.js and put back through the front door. A freshly-regenerated artifact failing its own freshness check on the day it was made is the shape of a check that gets filed as *known*, and CLAUDE.md names normalisation, not invisibility, as how the docs-currency guard rotted.

The unit that answers the real question is absolute power, and provenance.js **now prints it.** Every drift note carries a POWER line beside it:

- ci_gain — how many percentage points narrower the 95% interval would be. Precision goes as $1/\sqrt{n}$, so $1.96 \times 0.5 \times (1/\sqrt{n_{\text{dec}}} - 1/\sqrt{n_{\text{now}}})$.

- `max_shift` — how far the pooled point estimate could move if every missing game arrived. The pooled mean shifts by $(m/n_{\text{now}})(\bar{x}_{\text{new}} - \bar{x}_{\text{old}})$ and $\text{se}(\bar{x}_{\text{new}}) = \text{sd}/\sqrt{m}$, so a 2sd bound is $2 \times 0.5 \times \sqrt{m} / n_{\text{now}}$. Worst case, not expected case.

Measured across all thirteen drifting artifacts, the percentages span 4× and the power spans 2×:

artifact	drift	missing	CI gain	max shift (2sd)
<code>war.json</code>	48.6%	3,460	0.46 pts	0.83 pts
<code>policy-eval.json</code>	45.2%	3,220	0.41	0.80
<code>pory-eval.json</code>	35.1%	2,500	0.28	0.70
<code>xatu-belief.json</code>	31.1%	2,213	0.24	0.66
<code>guru-matchups.json</code>	26.1%	1,858	0.19	0.61
<code>roles-eval.json</code> and family	26.0%	1,854	0.19	0.60
<code>pory-nn.json</code>	15.7%	1,115	0.10	0.47
<code>counterplay.json</code>	12.8%	915	0.08	0.42

No artifact in this repository has enough missing data to move a proportion by one percentage point. `war.json` is missing *half its corpus* and can move 0.83 points. The smallest split-half noise floor this division has published is **0.43 points** (R1's cuts run 0.43–2.01; R4's three run 0.2 / 1.3 / 3.9), so `counterplay.json` is already **below the noise floor** and the rest sit inside a factor of two of it. "24% behind" and "the games it lacks cannot move it past its own noise floor" are different statements and only the second one is actionable.

And it self-extinguishes, which is the property the percentage lacks. `max_shift` = $\sqrt{f}/\sqrt{n}$, so the same 15.7% drift that moves 0.47 points at $n=7,123$ moves 0.33 at $n=14,000$ and 0.24 at $n=28,000$. The treadmill stops on its own as the corpus grows instead of being switched off by hand.

What was NOT changed, deliberately: the 10% trigger. Two reasons, and the second is the honest limit of this work.

1. Lowering the bar changes thirteen artifacts' status and that is not a call to make inside a measurement pass.
2. `max_shift` **still cannot see the thing that decided every row of §5c's hand triage** — the DISTANCE from an artifact's headline estimate to its decision boundary. `roles-eval.json` publishes 0.6935 against a coin's 0.6931; that 0.0004 margin is flippable by any new data at all, while `war.json`'s null is not flippable by 0.83 points. That margin is not computable from `n`, and the artifact is the only thing that knows it. `max_shift` is also stated for a **proportion** at $\text{sd} = 0.5$; a log-loss lives on another scale and the number is not directly comparable there. The next rung is a declared `decision_margin`, in the same convention as below.

A grace period measured in regenerations is rejected. It is a clock with extra steps, it cannot tell `chomp-ev.json` from `pory-nn.json`, and a fixed window is a licence for a genuinely flippable artifact to sit quiet inside it.

The `pory-eval.json` false positive is fixed on the reader side and still needs its generator. `provenance.js` now honours a declared `population_ceiling` (`j.population_ceiling`, `provenance.population_ceiling` or `corpus.population_ceiling`) and measures drift against it — the same declaration convention as `not_store_derived`, `raw_store_ok`, `gate` and `games_requested`. `pory-eval.json` is a strict subset (clean ladder games whose raw log exists AND names a winner: 5,456, not 7,123), so it can never get below ~21% against the wrong denominator. **This is a separate defect from the unit question and the answer above does not fix it by itself:** the hatch exists, and `engine/pory.py` must write the key

on its next deliberate run. **DONE 2026-08-05** — the deliberate run happened (§5d addendum) and the generator now writes `population_ceiling` with a note naming its predicate. Every artifact reading `games.ladder.raw-logs.jsonl` has the same shape.

WHAT THE NEW RULE WOULD AND WOULD NOT HAVE CAUGHT, plainly.

- `data/counters.json` , **older than the quality filter, UNSAFE for nine days — CAUGHT, and it was never a drift case.** That is the `FILTER_MT` check: the PREDICATE changed, so the artifact answers a different question, and no amount of power makes it valid. It is untouched, it is still `bad` rather than `warn` , and it still fails `--strict` . Confusing the two checks is how a volume rule gets credit for a correctness rule's catch.
- `data/chomp-ev.json` **four days behind, publishing "does not beat a coin" about a model with a directional edge — CAUGHT, and better than before.** That verdict sits *at* its boundary, so its decision margin is ≈ 0 and any `max_shift` exceeds it. The percentage caught it at 28% > 10%; the power rule catches it for the right reason.
- **An artifact recording a corpus it did not use — NOT CAUGHT, by either rule, and this file already says so on every run.** Only re-running the generator can.
- `data/slowking-playstyle.js` , **a GURU run written under the playstyle name — NOT CAUGHT by anything, and this is why the crude mtime rule in `test-site-data-fresh.js` was left alone.** See §9.

6. The noise floor is not a standing artifact

Split one arm in half and measure the spread. An effect smaller than that is not an effect. This gets re-derived by hand every time somebody needs it, which means it usually is not derived at all.

Two consumers now emit their own and neither is general: `rollout-r4.json` carries three split-half cuts of the H2H, and every block of `winrate-backtest.json` carries a `noise_floor` on Brier and on accuracy. That is the right shape — the floor belongs to the measurement, not to a global constant — but there is still no A/A run for the H2H, and a floor computed inside the arm being judged cannot see between-run variance.

7. All four rungs carry `n_measured / n_unit` — CLOSED 2026-08-04

`engine/rollout_r1_artifact.js` and `engine/rollout_r4.js` now write the pair R2 and R3 already carried, and both artifacts were regenerated from committed evidence (no rollouts): `data/rollout-r1.json` **9,201 scored positions**, `data/rollout-r4.json` **535 decisive pairs**. Choosing which of R4's four numbers goes in the common slot is the whole point of having one — the SPRT is computed on decisive pairs and nothing else, and the handoff quoting "5,248 games" (the line count of a store that writes two lines per game) is what the slot is for.

Still **not** called `n` : `data/rollout-r3.json` has published `n` as the rollout BUDGET since 2026-08-03, and one key meaning a sample size in one rung and a budget in the next is worse than no common key.

`tests/test-rollout-gates.js` derives the rung list from the filenames `engine/rollout_r*.js` write, asserts every generator emits both keys, and then permits exactly one artifact state beyond "carries them": *its generator does, awaiting a re-run*. `data/rollout-cost.json` is in that state and cannot leave it here — it is a set of TIMINGS, and R2 is re-run or it is nothing. What the test forbids is the state that actually goes wrong: missing in the artifact **and** in the generator, which is nobody having done it.

8. Naive timestamps — CLOSED 2026-08-04, and it was FIVE writers, not one

`engine/rollout_r1_join.py` was the reported case. The real answer to "is one occurrence a typo or a pattern" is that `datetime.now().isoformat(timespec="seconds")` appeared in **five** generators — `rollout_r1_join.py` , `lookahead_bound.py` , `lookahead_clock_control.py` , `nmf_rank.py` , `polygon2.py` — which makes it the house style rather than a slip. Eight committed artifacts carry one, and all eight come from exactly those five.

Correct the diagnosis, not just the bug. JavaScript does not misparse it. ECMA-262 gives the two ISO forms opposite defaults — date-TIME with no offset is read as LOCAL, date-ONLY is read as UTC:

```
new Date('2026-08-03T04:14:10') -> 2026-08-03T08:14:10.000Z (local, this box is UTC-4)
new Date('2026-08-03')          -> 2026-08-03T00:00:00.000Z (UTC)
```

So the four-hour figure is the RENDERED string, not the parse, and on this machine the value round-trips. The defect is that the stamp means something different to every reader, that the two forms this project already uses side by side follow opposite rules, and that it is wrong by the reader's UTC offset the moment it is compared against a `Z` stamp — which is what every JavaScript writer here emits and what `status.js` and `provenance.js` exist to do.

`engine/isotime.py` is the single home (`utc_now()` , `utc_today()`); all five call it. `data/rollout-r1-withdrawn-join.json` is deliberately NOT regenerated — it is a withdrawn result kept so the withdrawal can be checked. `tests/test-timestamps.js` gates the WRITERS, asserts the two ISO forms really do disagree on the running machine rather than quoting a comment about it, and lists the artifacts still carrying a naive stamp without failing on them, because an artifact is fixed by re-running its generator and that pressure is how "KNOWN FAILURE" gets typed.

9. The two "stale bundles" — one was a no-op and the other was never the file it claims to be

Both were regenerated with the verify-before-trusting step first. That step is the entire finding.

`data/engine-data.js` — **BYTE-IDENTICAL. Nothing was landed.** `SHOWDOWN_PATH=...` `node build/rebuild_sets_from_sheets.js` reports 318 species, 195 rebuilt from real sheets, 123 left alone under 10 sheets, **materially changed 0, illegal abilities fixed 0**. Run with `--write` and diffed against a preserved copy: **identical to the byte**. The generator reproduces its own artifact and the 0.9-day staleness was `mtime` and nothing else.

The original `mtime` was then RESTORED, and that is the point of the entry. Writing the identical file moved `engine-data.js` forward and immediately turned `counterplay.json` , `scoreboard.js` and `winrate-backtest.json` — this division's own leaf-calibration artifact — into "*older than its input `engine-data.js`*". Three false staleness flags manufactured by a regeneration that changed nothing. A restamp with negative information content is still a restamp; `status.js` 's refit edge is hash-based (`feature_fixture --check`) and was never at risk, but `provenance.js` 's input-ordering rule is `mtime`-based and was.

REPAIRED AND LANDED 2026-08-04. The section below is kept as the diagnosis; this is the result. Run with **both** variables set — `TAG=playstyle` `MATRIX_FILE=data/playstyle-matchups.json` — every figure predicted below reproduces to the digit: `n_games` 5,265 → **2,860**, `archetypes` 12 → **8**, `mixture` → **Rain 0.8079 / Setup 0.1657 / FakeOutBalance 0.0255**, `greedy-Nash` 0.0409 [−0.0001, 0.1735] → **0.026 [−0.0001, 0.1498]**, `uniform` 0.0761 → **0.0338**, `triples` 1,320 → **336**, `cycle` → **TailwindOffense** → **Sand** → **TrickRoom** (legs on 40, **5** and 140 games, still `supported: false`), and **the verdict flips** to "no material exploitability gap ... close to transitive at this granularity." It reproduces itself on a second run, byte-identical, and is no longer byte-identical to `data/slowking.js` . **The GURU arm was re-run first and reproduces its own artifact bit-for-bit** — every shared key unchanged, which is what licensed trusting the `playstyle` run from the same code. **THE FIX IS THE DEFAULT, NOT THE FILE.** `engine/slowking_preview.py` now REFUSES to write a `TAG` -named artifact from the default matrix and prints the two-variable command. The rule is narrow on purpose: a `TAG` names a NON-default run, so `TAG-set-with-default-matrix` is the one combination that cannot mean anything; the ordinary GURU run and the correct `playstyle` run are both untouched. A relative `MATRIX_FILE` now resolves against the repo rather than the shell's `cwd` — the documented command only

worked from the repository root, and a path that works from one directory and not another is how the wrong matrix gets reached for. **A second half of the same bug, found on the way:** `source_matrix` was the hardcoded literal `"data/guru-matchups.json"`. So even a CORRECT playstyle run would have stamped the GURU matrix as its source — the one field that could have exposed the clobber was pinned to agree with it. It is derived now, and `tag` is recorded beside it. **What moves on the page** (`app/index.html:907-923` / `web/index.html`, WEB's files, not edited): games 5,265 → 2,860; the cycle legend from three species pairs to TailwindOffense → Sand → TrickRoom; leg edge 10% → 5%; the mixture chips from Gengar-Incineroar 66% / Charizard-Garchomp 22% / Pelipper-Archaludon 12% to **Rain 81% / Setup 17% / FakeOutBalance 3%**; greedy exploitability 4% → 3%. **And two TYPED literals in that paragraph are now wrong on both pages** — "these matchups rest on 49, 37 and 15 games" (really 40, 5 and 140) and "the strongest of 1,320 candidate triples" (really 336). **Worse than the numbers: the room's whole thesis is now contradicted by its own artifact.** The panel is headed "The meta looks like rock-paper-scissors" and argues "picking one single playstyle is exploitable while a mixture isn't — the reason to mix", while the artifact it renders now says mixing buys little here. That is a WEB pass, and it is a rewrite rather than a number swap. **Three consumers checked.** `engine/sanity_check.py` passes, and its §5 check "site mixture top == report top" now reads **Rain == Rain**; before the repair it compared two copies of the same wrong file and passed for that reason. `tests/test-docs-current.js` §1b likewise now reads the real playstyle artifact (0.026, CI [-0.0001, 0.1498]) where it had been reading GURU's numbers under the playstyle name — a guard built to track this artifact was tracking the other one. And `engine/build-status.js:18` reads `slowking-playstyle-eval.json` into a variable `ex` that **nothing in the file ever uses**; it is a consumer in name only.

`data/slowking-playstyle.js` — **STOP. It is not stale; it is the wrong file, and has been since 2026-08-03 15:15.**

`engine/slowking_preview.py` takes its OUTPUT NAME from `TAG` and its MATRIX from `MATRIX_FILE`, which **defaults to** `data/guru-matchups.json`. Run with `TAG=playstyle` and `MATRIX_FILE` unset it writes a GURU result under the playstyle name. Measured:

- `data/slowking-playstyle.js` has a payload **byte-identical** to `data/slowking.js`;
- `data/slowking-playstyle-eval.json` is a **byte-identical file** to `data/slowking-eval.json`;
- both read 5,265 games / 12 species-pair archetypes / 1,320 candidate triples — GURU's shape.
`data/playstyle-matchups.json` holds **2,860 games over 8 playstyles**.

Regenerating it correctly moves published figures, so it was **restored and not landed**: `n_games` 5,265 → **2,860**, archetypes 12 → **8**, mixture Gengar-Incineroar 0.66 / Charizard-Garchomp 0.22 / Pelipper-Archaludon 0.12 → **Rain 0.81 / Setup 0.17 / FakeOutBalance 0.03**, greedy-Nash 0.0409 [-0.0001, 0.1735] → **0.026 [-0.0001, 0.1498]**, uniform 0.0761 → **0.0338**, cycle Charizard-Garchomp→Kingambit-Garchomp→Incineroar-Whimsicott → **TailwindOffense→Sand→TrickRoom**, triples searched 1,320 → **336**, and the verdict string flips from "substantially less exploitable... the meta is non-transitive here (rock-paper-scissors)" to "no material exploitability gap between Nash and greedy — this meta is close to transitive at this granularity."

The corrected numbers are the ones `docs/MODELS.md` **already publishes** in the MACHAMP entry — 336 triples, a leg on 5 games, 0.026, [-0.0001, 0.1498] — to the digit. So the docs are right and the artifact is wrong, which is the rare direction, and the 2026-08-02 withdrawal of the SLOWKING cycle still rests on measured evidence. `engine/build-status.js:18` and `engine/sanity_check.py:32` both read the clobbered file, and `app/index.html:907-923` renders its mixture and cycle legs. The repair is one command with **both** variables set; it is a WEB pass, not a refresh. The generator should refuse to write a `TAG`-named artifact from the default matrix.

Two things this costs the checkers, and both are recorded rather than fixed here.

- `provenance.js` reports `slowking-playstyle.js` as `ok`, correctly by its own rules — it is co-generated with `slowking.js`, so the ordering carries no information. Provenance sees ordering and declared counts; it cannot see that a file's CONTENT came from the wrong input. This is why the crude mtime bundle rule in `tests/test-site-data-fresh.js` was **left alone** despite §5f: delegating it to drift today would have marked this file clean.
- `tests/test-site-data-fresh.js` printed the repair as `node engine/slowking_preview.py` — the wrong interpreter, in the STALE table only; the `--list` path already derived it from the extension. Fixed. The command it names is still incomplete, which the test's own comments already admit, and running it with `TAG` set and `MATRIX_FILE` unset is a plausible route to the clobber that is on disk.

10. `train_value.py` was discarding a fifth of the corpus — PRIORITIES #13, FIXED 2026-08-04

`idn()` normalises punctuation and nothing else, so the event stream's `charizardmegay` never matched the bring list's `charizard`, `side_of` returned `None`, and the event was thrown away without a word. **Measured on 4,000 clean games before the fix: 21.7% of faints, 22.7% of damaging events, 20.8% of all damage, at least one discard in 96.5% of games, and 97.6% of discarded targets are megas.** The visible symptom is the one to remember: **88.9% of clean games ENDED with both sides still holding bodies.** The value net was learning from trajectories in which almost nobody ever loses their team.

The fix is a verb on the one resolver, not a fourth copy of it. `engine/mc_key.js` gained `mcKey.base` (which body is this) and `mcKey.bases` (the whole map, for a caller that cannot call into JavaScript per name), reading the `base` field the generator already wrote from the dex into `MC.mons`. Three properties were deliberate:

- **It is not a string strip.** `re.sub(r'mega[xy]?$', '', s)` is the obvious three lines and it is wrong — `mc_key.js` already records that the identical JavaScript strip answered Victreebel for Victreebel-Mega. The table carries the answer; this reads it.
- **It returns a flat BODY id, not a table key.** `MC.mons` holds `floette-mega` with `base: "floette"` and holds no `floette` row, so a version resolving the base back through the table returned null for exactly the 1,613 events this exists to rescue, one layer down. Being in our damage table is a fact about our table; the body is a fact about the game.
- **It touches no dex, so it cannot need `SHOWDOWN_PATH`** — the crash-instead-of-fail mode PRIORITIES #40 records for two other ratchets. `train_value.py` shells it once per run and **fails loudly** if it cannot, rather than reverting to the behaviour above.

After: 22.7% → 1.7% of damaging events dropped, 21.7% → 1.5% of faints, and games ending with both sides intact 88.9% → 26.3%.

What it moved, and the honest size of it. Paired on identical held-out states — 1,445 games, 10,120 states, both arms fitted on the same split:

	before	after	paired difference
log-loss	0.6634	0.6520	-0.0114, 95% CI [-0.0183, -0.0041] (bootstrap clustered by game)
accuracy	59.72%	61.47%	+1.75 pts, 95% CI [0.50, 2.94]

The mechanism is legible in the weights: `hpDiff` moved **0.169 → 0.377**, because a fifth of all damage had never been applied and the feature was attenuated toward zero. The shipped artifact (ladder + self-play, as `main()` defaults) moved `test_logloss` **0.6638 → 0.6536**.

Both intervals clear zero and the effect is still inside the noise floor for an unpaired comparison. Twenty split-half cuts of the fixed arm alone spread by a **median 1.87 accuracy points** (range 0.30–5.37) against a 1.75-point effect. The pairing is what buys the resolution; two runs on different samples could not tell these value nets apart. And the ceiling is unchanged — 61.4% sits below the **66.92%** in-sample ceiling for this feature class and below the live leaf's **67.97%**. **This is a correctness fix, not a capability change, and it was worth making on the first ground alone.**

Residual, measured rather than assumed: 1.7% still drops, and 1,613 of the 1,625 are one species. `MC.mons` carries `floette-mega` → `floette`, the store's bring lists hold `floetteeternal`, and no `floette` row exists — the chain does not close. The rest are in-battle formes the mega table does not cover (`mimikyubusted` 274, `morpekohangry` 48, `castformsnowy` / `castformrainy` 11). **Filed to ENGINE, not patched here:** closing them means reaching for the Showdown dex, which would make `train_value.py` produce different numbers depending on whether `SHOWDOWN_PATH` is set. That is *fitting environment and playing environment must match* in a new place, and it is not worth 1.5% of events.

11. THE THIRD COPY OF THE WEATHER MAP IS IN `board.js`, IT IS WRONG, AND THE FIXTURE CANNOT SEE IT

LANDED 2026-08-04, with the two fixture boards it needed, and refitted. The diagnosis below is kept because it is the reason the fixture grew; the result is here. `engine/board.js` no longer keeps a weather map. Both reads — `dmgFractions` and the `punishExposure` call in `featuresFor` — go through a `weatherKind(board, D)` helper that calls the damage engine's exported `weatherId`, the same consolidation ENGINE made in `tag_dex.js`. A damage engine that cannot answer is counted in `dmgFailures.weatherUntranslated` rather than defaulted to clear skies; measured over 234,873 candidate vectors it is **0**, on both the node and the browser export path (`tests/test-board-browser.js` : 58 of 58 features agree to 6 dp). **THE PRE-LANDING MEASUREMENT REPRODUCES ON THE CURRENT ENGINE TO THE ROW.** Re-run before relying on it, because the engine had moved underneath it: `fit_policy.decisionsFor` over the first 1,200 open-sheet corpus games, **32,054 decisions / 234,873 candidate vectors**, one process holding both builds of `board.js` : | | measured 2026-08-04 (pre) | re-measured after the ENGINE band | |---|---|---| | candidate vectors that move | 1,768 (0.75%) | **1,768 (0.75%)** | | decisions that move | 892 of 32,054 (2.78%) | **892 (2.78%)** | | columns that move | 14 of 58 | **14 of 58** | | games containing a moved vector | not measured | **238 of 1,200 (19.83%)** | The 19.83% is the new number and it is the one that matches the census: 18.5% of games carry sand or snow at some turn. **THE FIXTURE NOW SEES IT — 0 columns before, 10 after.** `sand-is-up` and `snow-is-up` join `SCENARIOS`. `Tyranitar` takes special hits under sand (the Rock special-defence 1.5x) and `Ninetales-Alola` and `Weavile` take physical hits under snow (the Ice defence 1.5x); `Hippowdon` and `Ninetales-Alola` both carry Weather Ball, whose TYPE resolves off the same field, so a translation that mapped one weather and not the other cannot pass both. A Rock body and an Ice body sit on each bench, because the switch family prices a body that is not on the field and would otherwise be untouched — that one choice took the detection from 6 columns to 10. Scored against the pre-fix map: `koTarget`, `dmgFrac`, `killIsRoll`, `killsThreat`, `koFirst`, `switchSurvives1`, `switchKOFast`, `switchDiesFirst`, `benchRisk`, and the joint `partnerCoversMe`. **State the limit rather than the win: the fixture catches 10 of the 14 columns the corpus moves.** `protectThreatened`, `diesBeforeMoving`, `screenValue` and `switchKOSlow` move on corpus boards and not on these ten. A fixture is evidence for the restamp rule, never proof of it, and this is the measured size of the gap rather than a caveat in prose. Coverage did not regress: 40 slots, 324 candidates, 1,309 pairs, and **0 features that never fire**. Two properties of the landing worth keeping. `weatherId` still does not know `desolateland` / `primordialsea`; on 339,483 corpus turn-boards that costs nothing and adding them is ENGINE's call on

SD2WEATHER , not a fourth table here. And adding scenarios re-stamps every hash, so it went in the same pass as the refit — a fixture change and a restamp cannot be separated.

This is a refit trigger. It is measured, it is NOT landed, and the gate for landing it is the Po/P1 band, which is not met.

engine/board.js:1190 carries WEATHER_KIND , a third private copy of the Showdown-weather → engine-weather translation that medicham2-browser.js owns as SD2WEATHER / weatherId . It is read at two sites — dmgFractions (:1247 , every damage-derived MAG feature) and the punishExposure call in featuresFor (:2937 , clickCost). What it holds:

```
{ sunnyday: 'sun', desolateland: 'sun', raindance: 'rain', primordialsea: 'rain' }
```

It maps the two weathers this format cannot produce and misses the two it does. desolateland and primordialsea are primal weather: **0 occurrences in 339,483 corpus turn-boards.** sandstorm and snowscape are not in the table at all, so WEATHER_KIND[board.weather] is undefined , || '' makes it clear skies, and **every damage feature under sand or snow has been computed in no weather.** The engine reads field.weather === 'sand' for the Rock special-defence 1.5×, === 'snow' for the Ice defence 1.5×, and Weather Ball's type comes off the same field.

Exposure, re-measured rather than inherited — a census of board.weather at every turn-board across games.ladder + games.bo3 + games.ots (52,441 games):

weather at a turn-board	share	WEATHER_KIND gives
clear	64.15%	'' correct
sunnyday	14.90%	'sun' correct
raindance	10.23%	'rain' correct
snowscape	5.43%	'' — WRONG
sandstorm	5.29%	'' — WRONG

10.72% of turn-boards, and 18.5% of games contain at least one.

THE FIXTURE PASSES ANYWAY, AND THAT IS THE FINDING. The patch was applied, measured and reverted. engine/feature_fixture.js --check returns feature semantics OK on both policy-weights.json and policy-weights-joint.json **before and after** — because the fixture's only two weather scenarios are RainDance and SunnyDay , the two WEATHER_KIND already gets right. All 58 columns are hash-identical while the feature function has moved.

What actually moves, measured on the fit's own rows — fit_policy.decisionsFor over the first 1,200 open-sheet corpus games, **32,054 decisions / 234,873 candidate vectors:**

candidate vectors that move	1,768 (0.75%)
decisions that move	892 of 32,054 (2.78%)
columns that move	14 of 58

dmgFrac (1,182), koTarget (238, max |Δ| 0.93), killIsRoll , killsThreat , diesBeforeMoving , benchRisk , koFirst , protectThreatened , switchKOFast , switchKOSlow , screenValue , switchDiesFirst , switchSurvives1 , switchSurvives2 — several flipping a full 0→1.

So feature_fixture --check is necessary and not sufficient, and status.js 's refit edge: CLEAN inherits that limit. The hash covers the boards the fixture builds; the feature FUNCTION lives over every board the corpus contains. This division's own rule — *a restamp is only valid if the feature FUNCTION is*

unchanged — is the binding one, and a green fixture is evidence for it, not proof of it. **The fixture needs a sand board and a snow board.** Adding them is itself a fixture change that re-stamps every hash, so it belongs in the same pass as the refit, not before it.

The patch is small and is recorded here rather than left on disk: replace both reads with a `weatherKind(board, D)` helper that calls the damage engine's exported `weatherId`. Do not write a fourth map. Note that `weatherId` does not know `desolateland / primordialsea`; on the measured corpus that costs nothing, and adding them is ENGINE's call on `SD2WEATHER`, not a second table here.

11a. `position_features.js` — the same defect at the second boundary. LANDED, no refit owed.
`engine/position_features.js:292` built its field object as `B.norm(board.weather || '')` — the board's *move name* — and handed it to `M.dmgRange` and `M.effSpeed`, which compare against the engine's words. Truthy and meaningless, exactly as at the leaf boundary. Now `M.weatherId(...)`.

No refit is owed and that was checked, not assumed: nothing in the repository fits, reads or renders these columns. The only callers of `positionFeatures` are four tests, and no artifact contains any of its feature names.

Exposure re-measured on the boards this module is actually asked about — the `joint_rows.build` walk, 400 open-sheet games, **3,202 mid-game boards / 6,404 scored positions. 49.94% carry a weather**, higher than the 35.85% turn-board figure because weather accumulates as a game runs.

- **1,962 of 6,404 positions moved (30.64%):** sun 73.6%, rain 66.7%, sand 49.5%, snow 29.5%.
- 7 of 16 columns: `raceEdge` (29.67%, max $|\Delta|$ 0.42), `killFirstEdge`, `iKillNext`, `theyKillNext`, `benchAnswersDiff`, `speedEdge`, `pinnedDiff`.
- **0 of 3,206 clear-weather positions moved** — the control that says this is the weather.

`:296` 's terrain now goes through `M.terrainId` as well. That one is a **confirmed no-op**: 16 of the 3,202 boards carry a terrain, 11 of them under clear weather, and all 22 positions scored on those are bit-identical. Every downstream reader already calls `terrainId` itself. It is translated here so the field object leaves in one vocabulary rather than two, which is the condition that let the weather half go unnoticed. The four probed keys are a list of BOARD KEYS, not a translation table — the same justification `rollout_leaf.terrainOnBoard` records.

ENGINE's "*o of 400 boards*" for this file was terrain-only and is reproduced (0.50% here). It was being read as though it covered the weather too, and the weather number is 49.94%.

11b. `git checkout -- <file>` INVALIDATES EVERY CONTENT STAMP ON THIS MACHINE

Found by doing it. `core.autocrlf=true`, the committed blobs are LF, and the worktree files are LF — so a checkout REWRITES them as CRLF. The bytes change, nothing in git notices (`git diff` is empty), and `sha256(worktree)` moves. `winrate-backtest.json` 's `measured_against` and `run_stamp.js` 's `source_digests` both hash worktree bytes, so `engine/board.js` immediately began printing **PRE-CHANGE** — measured against a different build of: ... `engine/board.js` after a revert that changed no code. Converted back to LF; the digest returns to `bcf2dab9dc6f`, which is the stamp exactly, and `board.js` drops out of the **PRE-CHANGE** list.

This is the mirror of the warning already in *Reading a stamp* — *never compare `source_digests` to `git.blobs`, they differ by line-ending translation*. The new half is that an ordinary git operation can move one of them. **After any `git checkout --` or `git stash pop` on this box, check `node engine/status.js` before believing a **PRE-CHANGE** line.**

12. tests/test-no-silent-failure.js — the 32 MEASURE entries, and what two of them were hiding

Worked through without `--update`, which would have laundered the SEARCH, OPS and WEB entries in the same command. **NEW since the baseline: 52 → 20.** Every MEASURE-owned entry is cleared; the 20 that remain are 13 SEARCH (`miltank.js`, `rollout_leaf.js`, `rollout_r1.js`), 3 OPS (`mag_bot.js`) and 4 in `tests/test-{site-data-fresh, stadium-roster, web-parses}.js`.

Two were hiding something real.

`engine/run_stamp.js:92` recorded **"the tree was clean" whenever git refused to answer.**

```
const porcelain = git(['status', '--porcelain', '--'].concat(watched)) || '';
```

`git()` returns `null` when the command throws — an index lock, an interrupted rebase (CLAUDE.md documents this repository reaching one 43 commits into a 45-commit replay), `git not on PATH`. An EMPTY porcelain is git's way of saying CLEAN. `|| ''` collapsed the two, so a stamp written while git was unavailable published `dirty: false` **beside a commit id that described nothing on disk** — and this module's own header says *"a clean commit id over a dirty tree is a lie of exactly the kind this module exists to stop"*, while `docs/MEASURE.md`'s *Reading a stamp* tells every reader to trust the commit when `dirty` is false. `rev-parse HEAD` was already guarded; `status --porcelain` was not. Now a third state: `dirty: null` with `git_errors`, and `status.js` renders it as **DIRTINESS UNKNOWN** rather than as the clean case.

`engine/backtest_winrate.js:71` + `engine/status.js:176` **composed into a false clean bill.** `stampOf` returns `{mtime: null, error}` with no `sha256_12` when a source cannot be read. `status.js` then did `if (!st || !st.sha256_12) continue`; and, finding nothing in `moved`, printed **"CURRENT — every engine source the leaf reads still hashes to what it was measured against"**. With every stamp failed, that sentence was printed over **zero comparisons**. Two silent catches, neither wrong on its own, producing a clean provenance line on this division's headline number. The count is now stated (`all N engine sources`), unstamped sources are named, `NOT DERIVED` is printed when N is zero, and a source that has been DELETED is reported as gone rather than as "a different build of".

The rest were latent rather than active, and the honest answer is that they were hiding nothing today — which is a measurement, not an absence of one.

- `engine/rollout_r4.js:279` — the split-half scan that produces the NOISE FLOOR discarded a torn line in silence, while `countLine`, reading the same file for the header counts, keeps `torn` and publishes it. A row lost here shrinks an arm and moves the spread, and the spread is the entire output. Counted; the split now refuses to report if anything was lost. **Measured on `games.r4-decided.jsonl`: 0 torn, 0 bad-seed, across all three cuts.** Before the counter existed, 0 and 500 looked identical from there. A second hole found while adding it: a non-numeric seed put every such record on side B, because `NaN % 2 !== 0` — now rejected rather than piled up. Incidental: the `seed hash parity` cut splits **1,382 / 1,242**, an 11% imbalance, and it is the cut that produced the largest of the three spreads (3.9 pts) quoted as this run's noise-floor range.
- `tests/test-timestamps.js:49/54` — a directory that would not list and a file that would not read were both skipped with `continue`, so *"no Python generator writes a naive datetime"* was also the answer when **zero generators had been looked at**. That is CLAUDE.md's *a capability that cannot prove it ran* inside a guard written for a different failure. Now asserts a floor on files scanned (39 in `engine/`, 1 in `build/`; `tools/` has no `.py` and `scripts/` does not exist) and fails on anything skipped unread.
- `tests/test-timestamps.js:92` — an artifact that would not parse was silently excluded from the published *"N artifacts still carry a naive stamp"* list, so the files most likely to be broken were the ones the survey

could not see. Now counted and named: **0 of 108** `data/*.json` **fail to parse**, so the list of 8 is complete — a statement that could not previously be made at all.

- `tests/test-web-status.js:181` — `catch { return false }` on the freshness filter means "not newer than the board", the same answer a perfectly fresh artifact gets. A source that had been **deleted or renamed** read as up to date, in the test whose job is that every rendered figure traces to an artifact. Missing is now its own failure. None are missing today.
- `tests/test-rollout-gates.js:81` and `engine/rollout_r1_artifact.js:228` — both collapsed "no such file" into "will not parse". The first then granted a CORRUPT gate artifact the one tolerated state ("*awaiting a re-run*"); the second made a broken sidecar indistinguishable from a run nobody ever stamped, which is the exact distinction §4 and §7 exist to preserve.
- `engine/status.js` (9), `engine/rollout_explore_sweep.js` (3), `engine/rollout_r1_artifact.js` (3 more), `engine/run_stamp.js:60`, `tests/test-web-status.js:58/112`, `tests/test-guru-derived.js:56` — each conflated *absent* with *unreadable*. `status.js` now carries a `DIAGNOSTICS` block, printed on screen and deliberately **outside** the section bodies so `--write` never stamps a transient into a ledger.

Two defects in the ratchet itself, filed not fixed:

- `--update` **is all-or-nothing, so the tool's own guidance cannot be followed.** It says "*if a silent fallback is genuinely right here, say why in the code and re-baseline with --update so the exception is deliberate and visible*" — but `--update` re-baselines every silent catch in the repo, including other divisions'. There is no way to bless ONE. It needs a per-entry allow with a reason string, in the shape `build_guru.js.js` 's `DELIBERATELY_UNUSED` already uses.
- `isSilent` **cannot see a recorder it does not recognise by name.** `error: e.message` inside a returned object is a colon, not an `=`, so an artifact that carries its own reason still reads as silent; and a named helper that pushes onto a list looks like nothing from inside the catch body. Four surviving entries are this. Widening the regex would launder real ones, so the code was moved to the documented convention instead — `status.js` 's recorder is named `logUnreadable`, and two locals are named `errWhy` / `errBundle`.

`--all` was added to the ratchet: the 25-line cap is right for a gate, but "*... and 27 more*" is how the tail of a list stops being anybody's job.

13. THE REFIT RAN — and it moved nothing measurable. That is the result, not a preamble to one.

`node --max-old-space-size=4096 engine/fit_policy.js` then `engine/fit_joint.js`, on the weather landing in §11. **8,759 clean open-sheet games, 229,339 usable decisions** (up from 8,414 / 220,613), 183,679 train / 45,660 held out, lambda selected on held-out likelihood at 0. Both weight files carry a fresh `featureHashes` over the 10-scenario fixture, and `feature_fixture --check` exits 0 on both.

The before/after in the artifacts is not the comparison to read, because the two fits have different corpora and different held-out sets — 44,033 decisions against 45,660. Quoting `heldOut.boardAware` 32.269% against 32.271% would be comparing two samples, which is the confound this division keeps finding in other people's work. The comparison that means something scores the SAME held-out decisions three ways, with the split reproduced exactly (`hash(game) % 5 === 0`):

arm	what it is	logL/decision	top-1
A	old weights + old features	-1.732548	32.204%
B	old weights + NEW features	-1.732200	32.252%
C	NEW weights + NEW features — what ships	-1.732276	32.178%

1,772 held-out games, 46,162 decisions, paired per decision, bootstrapped over 10,000 resamples of GAMES (decisions inside one game share a team and a board):

paired difference	logL/decision	top-1 points
B – A the weather fix alone, weights frozen	+0.000348 [0.000075, 0.000623]	+0.048 [0.009, 0.093]
C – B the refit, given the fixed features	–0.000076 [–0.000172, +0.000021]	–0.074 [–0.155, +0.004]
C – A everything, against what shipped	+0.000273 [–0.000010, +0.000556]	–0.026 [–0.117, +0.064]

Read plainly, three statements:

- 1. The correctness fix is detectable and it is a quarter of the noise floor.** B – A clears zero on both metrics, and twenty split-half cuts of arm C alone spread by a **median 0.192 top-1 points** (range 0.005–0.770) against an effect of 0.048. The pairing is the whole reason it resolves at all; two runs on different samples could not tell these builds apart. Same shape as §10's value net, one order of magnitude smaller.
- 2. Refitting the weights on the corrected features bought nothing.** C – B contains zero on both metrics and its point estimate is NEGATIVE. Only **1 of 58 weights moved more than 2 SE** (`dmgFrac` +0.0592, 2.45 SE — the column the fix touches most), 6 moved more than 1 SE, and the L2 norm of the whole weight change is **0.216**. The joint file moved less: largest term `terrainSetupHelpsPartner` +0.102, L2 **0.128**.
- 3. The combined change is indistinguishable from zero on held-out human-click prediction.** C – A contains zero on both. The fix was worth making because it is a fact about the game that the feature function was getting wrong on 10.72% of turn-boards — that is the whole justification and it does not need a metric to support it.

What this does NOT say. Top-1 agreement with a human click is not a win rate. Nothing here measures whether MILTANK plays better; that is an H2H and it belongs to SEARCH. And the leaf is a separate model from MAG — §1's finding that the leaf is worse than a coin is untouched by any of this.

13a. THE FIT IS NOT RUN IN THE ENVIRONMENT THE BOT PLAYS IN, and it is 20x the weather defect

This is the headline of the refit, not a caveat on it. CLAUDE.md's rule is *fitting environment and playing environment must match*, recorded because MAG's weights were once fitted with the sheet visible while the bot played without it. **The mismatch is back, pointing the other way, and nothing was watching for that direction.**

```
engine/fit_policy.js:376   board.setSheet(side, m.species, { nature, item })
engine/magnemite.js:522   this.board.setSheet(m[1], sp, { nature, item, ability, moves })
```

`Board.switchIn` copies all four onto the active mon, and `dmgMon`, `effAbility` and `movePriority` read them. So the LIVE player sees a sharper board than the fit ever did: the fit prices every opponent on the dataset's representative moveset and on Smogon's per-species ability odds, while an open-sheet game hands the player the declared four moves and the declared ability.

The sheets carry it. 14,400 sheet entries over 1,200 corpus games: 100.0% declare an ability, 100.0% declare four moves. This is not information the fit lacks — it is information the fit is handed and drops.

Measured the same way §11 was, one process holding both builds of `fit_policy`, identical games and identical decisions:

	weather defect (§11)	the sheet-channel gap
candidate vectors that move	1,768 (0.75%)	37,460 (15.95%)
decisions that move	892 (2.78%)	16,177 of 32,054 (50.47%)
columns that move	14 of 58	20 of 58
games containing a moved vector	238 (19.83%)	1,197 of 1,200 (99.75%)

switchDiesFirst (10,013), diesBeforeMoving (9,466), switchSurvives1 (6,632), dmgFrac (4,188), killsThreat , switchKOSlow , switchSurvives2 , switchKOFast , protectThreatened , priority (1,958 — the declared ability reaching movePriority), screenValue , movesFirst , koTarget , benchRisk , killIsRoll , koFirst , clickCost , passTurnAccrues , switchFaster , deadNoLastMove . The choice set is unchanged — the row counts match game for game — so this is purely what the board KNOWS, not what it offers.

It is NOT landed and no second refit was started. Landing it is a one-line change to fit_policy.js:376 plus a full refit of both files, and it would invalidate the refit reported above on the day it was published. It also needs a question answered first that this measurement does not answer: the fit's decisions come from games where the sheet was public, but MAG must also play the ~half of ladder games where the opponent declines OTS, and a model fitted on four channels degrades differently from one fitted on two when a channel goes missing. That is the Focus Sash lesson — *replacing a hedge with a certainty is only an improvement if you also track what invalidates the certainty* — and it is a decision, not a refresh.

Filed with its size stated, which is the part that was missing: **half of every decision the fit trains on is priced against a board the player does not see.**

13b. THE JOINT LAYER IS REFITTED ON FOUR CHANNELS, AND THE CHANNELS ARE WORTH A LIKELIHOOD GAIN, NOT AN ACCURACY GAIN — 2026-08-05

The other half of §13a's debt, run under Will's go. Three results, each with its instrument named.

The joint refit (engine/fit_joint.js , four-channel joint_rows.js): 8,856 clean open-sheet games, 101,459 joint turns → 95,886 usable, 77,975 train / 17,911 held out by game, lambda 0 on held-out. The artifact now carries a fitEnvironment block and it says matches_player: true by measurement: the declared ability and moves reached the board on **202,343 of 202,918 scored slots (99.7%)** and **395,130 of 396,288 live foe actives (99.7%)**. Held out, predicting the pair: separate decisions logL -3.3294 / top-1 10.3%, refit with joint terms zeroed -3.3199 / 9.8%, with the joint terms **-3.2308 / 12.2%**. The chosen pair fell outside the top-6 menu on 11.1% of kept turns. feature_fixture --check passes on the new artifact. No before/after against the presheet joint vector is quoted because none was measured — the presheet run published no held-out table to its artifact, and comparing two logs would be comparing two samples.

What the two extra channels are worth at the marginal layer (engine/sheet_channel_value.js , arm A = release d3d04b669e18 's two-channel incumbent, 44,982 paired held-out decisions over 1,789 games, 10,000 game-bootstrap resamples). The first run **VOIDED itself** — ENGINE saved engine/medicham2-browser.js mid-run and the instrument recorded void: true — and the second run is clean, with every deterministic figure identical between the two:

paired difference	logL/decision	top-1 points
B – A the information alone, weights frozen	+0.002853 [0.001611, 0.004072]	+0.009 [-0.140, +0.157]
C – B the refit, given the information	+0.002234 [0.001638, 0.002831]	+0.165 [0.029, 0.299]
C – A everything vs what shipped	+0.005087 [0.003854, 0.006331]	+0.173 [-0.011, +0.360]

Split-half noise floor of the shipping arm, 20 cuts: **median 0.331 top-1 points** (range 0.012–1.385; the earlier refit's floor was 0.192 on a smaller paired set). Read plainly: **the sheet channels buy a real likelihood gain — every logL interval clears zero — and no demonstrable top-1 gain.** The one top-1 interval that clears zero (C – B, +0.165) is half its own noise floor and resolves only because the comparison

is paired; the total effect against what shipped contains zero. Same shape as §13: correctness and information first, metric second, and the honest metric statement is "better calibrated per decision, not measurably more often right on the argmax."

The degradation budget did not move, and cannot move by this lever. `fit_joint.turnsDropped` is **5.4929% (5,573 of 101,459) against a 5.49% ceiling — still red**. The dropped turns are unmatched clicks (5,555) and ambiguous mirrors (18); the chosen pair is kept regardless of its rank, so the four-channel `w1` changes which ALTERNATIVES are on the menu, never which turns are kept. The rate crept from 5.4811% when the ceiling was ratcheted (86,242 turns) because the newly ingested games unmatch at 5.56%. The ceiling is untouched; the call on it is Will's.

PORY family regenerated, and `tests/test-site-data-fresh.js` is GREEN (7/7) — §5d addendum has the pory-eval numbers; `data/nmf-roles.json` moved 13,258 → 14,808 team-docs, 258 → 263 moves, recon-err 0.8346 → 0.8356, rank 10 unchanged, and both site bundles (`data/pory.js` , `data/nmf.js`) were rewritten by their own generators in the same runs. `data/pory-nn.json` retrained at 6,289 games / 106,782 states (was 6,008 / 102,296): every arm ordering holds — N6 0.6201, NR 0.6132 and LR 0.6064 all still beat the two-feature bar at 0.6229, nonlinearity is still worth ~0.003 and representation ~0.016 — and no living doc quotes these figures (§5c). The first retrain immediately re-red the drift check at 15.1%, the §5f false-denominator class to the letter: its population is the raw-logs subset. Both `engine/pory.py` and `engine/pory_nn.py` now declare `population_ceiling` (the artifact's own generator wrote it; the retrain reproduced every arm to the digit under its seeds), which is what turned the check green rather than a threshold being moved.

Doc and site locations quoting superseded PORY figures, for the propagation pass (grep-verified, historical HANDOFF files excluded as history): `docs/ABRA-whitepaper.md:113` (0.6298 [0.6125, 0.6456]), `docs/SUMMARY.md:77` (same + 0.655), `docs/MODELS.md:358/360/364` (0.9943/1.4080, 0.629799/0.629778, +0.000021 [-0.000013, +0.000056], 925, 4,623), and WEB's `web/stadium.html:506,:728` + `app/stadium.html:506,:728` (the `kadabra` data object and its prose), which render every one of those numbers and are flagged, not edited.

14. THE OUTPLAYED TURNS — 1,336 recorded actions were not clicks, and the model was learning from every one of them. LANDED 2026-08-05.

`docs/CLICK-CENSORING-FIX.md` is the spec, ordered by Will: *"i def dont like just tossing turns because they got outplayed with a move liek encore or follow me, these are the basis of vgc man."* Four artifacts: `data/click-censoring-census.json` , `data/partial-label-em.json` , `data/censoring-value.json` , and the refitted `data/policy-weights{,-joint}.json` .

LEAD WITH THE RESULT, INCLUDING THE HALF THAT DID NOT WORK. Two headline classes were measured and only one moved:

held-out class	what changed, after – before	verdict
COERCED (n=284) — Encore replaced the click, or the mon was dragged in	P(model picks the action no human chose) -0.002614 , 95% CI [-0.003663, -0.001637]	the poison is unlearned, and it is the only headline that moved
PARTIAL (n=643) — a redirector soaked the attack	mass on the true candidate set +0.000109 [-0.000286, +0.000491] ; logL on the set -0.002646 [-0.004037, -0.001377]	no improvement. The likelihood is very slightly WORSE.
CONTROL, CLEAN (n=46,268)	logL +0.000447 [0.000142, 0.000743] ; top-1 +0.002 [-0.094, 0.098]	as the spec predicted: no top-1 change

47,195 paired held-out decisions over 1,809 games, 10,000 bootstrap resamples **clustered by game**, `engine/censoring_value.js` . The spec disclaims a corpus-wide top-1 improvement in advance and none is claimed here; the CLEAN row is a control.

RE-MEASURED 2026-08-05 on the current engine and a corpus grown to 9,230 games — every figure in this section reproduces inside its interval. The table above is the 3.42.0 run and is kept as published; the artifact on disk now holds **n=48,274 over 1,851 held-out games**, **COERCED -0.002613 [-0.003650, -0.001672]**, **PARTIAL mass +0.000122 [-0.000261, +0.000514]**, **CLEAN logL +0.000485 [0.000189, 0.000777]**. §17 has the full comparison and the reason the engine move could not have touched it.

Say the negative result plainly: Stage C bought nothing measurable, and the reason was predicted by Stage C's own validation before the refit ran. The EM harness recovers **97.4%** of a planted censoring bias when the censoring is heavy, and at the rate the corpus actually censors, the bias in weight space is **-0.0030 against a 0.2600 noise floor** — unmeasurable. The redirection correction is right in principle, and the class is 1.35% of actions with a candidate set of exactly two, so there was almost nothing to recover. Both instruments agree, which is the only reason to believe either.

Stage A — the census. 241,927 recorded human actions over 8,942 clean open-sheet games (the FIT corpus; the census artifact has since been re-run twice with the store, at 9,022 and then **9,230** games, and the shares are stable to a hundredth of a point — see §17):

class	n	share	mechanism	
CLEAN	229,555	94.886%	—	
PARTIAL	3,260	1.3475%	Follow Me / Rage Powder 3,231; Lightning Rod 29. Every candidate set is size 2	
COERCED	1,336	0.5522%	Encore's application turn 1,116; a ` \	drag\ ` 220 (Roar 184, Dragon Tail 33, Whirlwind 3)
dropped, not a censoring class	7,776	3.214%	unmatched 6,937, trivial 809, ambiguous 30	

The mechanism list is read from the running format, never typed — moves with `condition.onOverrideAction` , moves with `forceSwitch` , abilities with `onFoeTryMove` , items assigning `switchFlag / forceSwitchFlag` (**empty here:** Eject Button, Eject Pack and Red Card are all `isNonstandard: 'Past'`), plus `data/tags.json`'s `redirects / redirectsType` . Every set refuses to be empty, and a zero on either counter is fatal in both fitters.

THE CLASSIFIER WAS SCORED AGAINST THE PROTOCOL, NOT ASSERTED. The census has a second arm that reads `data/games.*.raw-logs.jsonl` and compares per (game, turn, slot):

class	protocol says	classifier flagged	both	recall	precision
Encore application	619	642	617	99.68%	96.11%
drag	86	86	83	96.51%	96.51%

The 25 Encore false positives are 0.01% of all actions and the asymmetry is the right way round: a false positive deletes one real click, a false negative keeps a poisoned one, and there are two of those. Most likely cause is an Encore blocked by Protect, which the extractor records no failure flag for. Stated, not chased — the classifier was frozen while the refit that depends on it ran.

Two corrections to the spec, both measured. (1) §1's first row is wrong and `engine/redirect_audit.js` said so on 2026-08-02: redirection does **not** drop the turn. The redirector is a legal candidate target, so the matcher matches it and the click enters the fit with a CONFIDENT WRONG TARGET. It is label noise, not censoring — which makes Stage C a poison fix as much as a recovery. (2) A `\\|drag\\|` is a third coerced class the spec does not list. `engine/durable-ingest.js:67` parses `\\|switch\\|` , `\\|drag\\|` and `\\|replace\\|` with one regex, and `fit_policy`'s `forcedSlot` guard only knows about faints, so every phased arrival was fitted as a voluntary switch decision.

WILL'S FARIGIRAF CASE IS ANSWERED: PARTIAL, NOT ERASED.

```
|cant|p1a: Farigiraf|ability: Armor Tail|Aqua Jet|[of] p2b: Basculegion
```

The blocker is named first, the attempted **move** is named, and [of] names the **attacker. 284 of 284 priority-block lines carry the attacker slot (100.0%)**, so the user and the move are exact and only the target is ambiguous — between the blocker and its ally, and nowhere else, because the ability blocks nothing aimed elsewhere.

It is counted and NOT recovered, and that is a judgement with a reason. Showdown emits no `\|move\|` line for a blocked attempt, so the class leaves no event and lives only in the raw logs — which cover **66.17% of the fit corpus (5,917 of 8,942 games)**, and the gap is one SOURCE, `data/games.ots.json1`, an external archive with no log file. Recovering these 284 clicks, and the 126 more that `\|cant\|` states outright (Taunt 59, Disable 58, Heal Block 5, Imprison 4), would add outplayed turns from two stores and none from the third. That is a corpus reweighting wearing a bug fix's clothes. Closing it means re-ingesting the ots archive with its logs — OPS work, filed.

A FOURTH THING, FOUND ON THE WAY, AND IT IS A WRONG DENOMINATOR RATHER THAN A WRONG LABEL. `engine/board.js`'s `candidates()` narrows the choice set for a **Choice item**, derived from the dex's `isChoice`, with its own comment saying why: *"that is not a scoring error, it is a WRONG DENOMINATOR. A conditional logit divides by the sum over the choice set."* It does nothing about the other family that shrinks a menu — the `onDisableMove` set. **2,280 of 139,769 logged actions (1.6313%) were taken with a menu-sealing volatile up:** Encore 1,276, Throat Chop 375, Taunt 329, Disable 239, Heal Block 94. A human left one legal move by Encore is priced as having chosen it over nine. **NOT FIXED HERE** — narrowing the menu moves every feature row and owes its own refit, and it is a different defect from the one this dispatch was for. Counted so the decision has a size.

Stage C — the estimator, shown failing on known-bad input before it was believed.

`engine/em_validation.js`, 31,940 real corpus feature rows over 1,200 games with SYNTHETIC labels drawn from a known planted vector, 3 seeds, the real censoring process applied to the planted labels:

regime	rows censored	oracle	naive	EM	noise floor	verdict
amplified	20.961%	0.9978	1.8913	1.0208	0.2600	bias 0.8935 clears the floor; EM recovers 97.4%
observed	0.439%	0.9978	0.9948	1.0021	0.2600	bias -0.0030 — inside its own noise floor

Distances are $\|\hat{w} - w^*\|_2$. The noise floor is the spread of the ORACLE arm across the three seeds, so it carries no information about the contrast. The **first** amplified regime censored EVERY eligible row and EM recovered only 45% — correctly, because with every same-move row collapsed there is nothing left to identify the target features from. That is Cour et al.'s identifiability condition failing, not the estimator; the eligibility is now exogenous and the collapse label-dependent, which is what the corpus does. `engine/em_validation.js --check` re-verifies the recorded verdict AND re-hashes every source, so editing `engine/click_class.js` turns the gate red instead of leaving a stale PASS; it is registered in `tests/run-all.js`.

Stage D — what the refit moved, and the confound stated rather than buried. `data/policy-weights.json`: **8,942 games, 232,815 usable decisions of 241,927 seen** (186,494 train / 46,321 held out), lambda 0 on held-out, reweighted vector ships. $\|\text{new} - \text{old}\|_2 = 0.8030$ and **9 of 58 weights moved more than 2 SE**. The mechanism is legible in which ones:

feature	before → after	
deadStall	-1.3114 → -1.4763	5.44 SE
stallIntoEncore — "I am about to Protect and something across from me can Encore me for it"	-1.0502 → -1.6281	3.10 SE, the largest single movement

feature	before → after	
deadSide	-2.7606 → -3.1414	4.01 SE

That is the predicted direction. The poisoned rows were victims "choosing" their last move under an active Encore; deleting them makes clicking into an Encore threat look worse, and the Encore/stall family is exactly where the vector moved. Same shape as §10's `hpDiff` 0.169 → 0.377.

THE CONFOUND, NAMED: the two vectors differ in four ways, not one. The incumbent was fitted on 8,856 games and the new one on 8,942 — the collector never stops — so the Stage D contrast carries the coerced removal, the partial-label EM, 86 extra games and the refit itself. The weight-movement pattern above is evidence for attribution and is not proof of it. `CENSORING=off` now exists in `engine/fit_policy.js` for exactly this: it fits the OLD way on the NEW corpus, and it records `censoring: "off (CONTROL ARM – not shippable)"` in its own artifact so a control can never be mistaken for a ship. **That arm has not been run** — it is a second full refit and free RAM was 1.3 GB with the joint fit in flight. It is the next thing this section owes.

AND EVERY EFFECT HERE IS SMALLER THAN ITS OWN CLASS'S NOISE FLOOR. The COERCED contrast is 0.002614 against a split-half floor of 0.007635; the CLEAN logL gain is 0.000447 against 0.007855. They resolve only because the comparison is **paired per decision** — two runs on different samples could not tell these builds apart. This is the same statement §13 and §13b make, and it must travel with the numbers.

Stage B — the budgets are RE-DERIVED, not renumbered, and `turnsDropped` is retired. `fit_joint.turnsDropped` was `(turnsSeen – kept)/turnsSeen` and sat at 5.4929% against a 5.49% ceiling. Stages B–C change what "dropped" MEANS: coerced turns used to be inside `kept`, carrying a wrong label, and now leave the labelled set — so the old total would have gone UP while the artifact got strictly better, and a ceiling that may only tighten would have gone red for an improvement. **Raising or lowering the number would have been the wrong move in either direction.** Three counters now, each with its granularity stated in `data/degradation-budgets.json` and its ceiling ratcheted from a measured run:

counter	what it counts	denominator
<code>fit_policy.decisionsUnreadable / fit_joint.turnsUnreadable</code>	the click existed and could not be recovered. A LOSS. Successor to the old totals, and directly comparable because it is the same quantity minus a term that was misfiled as <code>kept</code>	human actions seen by <code>fit_policy</code> / joint turns seen by <code>fit_joint</code>
<code>fit_policy.coercedActions / fit_joint.coercedTurns</code>	the recorded action was not a click and was removed. A CORRECTION, not a loss — it should track the metagame's use of Encore and phazing and nothing else	same
<code>fit_policy.decisionsDropped / fit_joint.turnsDropped</code>	RETIRED. Carried in a new <code>superseded</code> block with its old ceiling intact, so the history is not deleted	—

`measured_at` also used to read "*over 120 corpus games*" on every row, which is true of the three `board.js` counters and **false of every fitter rate** — those come out of an artifact written over the whole corpus. A ceiling whose denominator is misdescribed cannot be re-derived by anyone.

What this does not say. Top-1 agreement with a human click is not a win rate; whether MILTANK plays better is an H2H and belongs to SEARCH. The COERCED class has no ground-truth label by construction, so its contrast measures a change in the MODEL, not an improvement in accuracy — it cannot be otherwise, and inventing an agreement number for it would have been the dishonest option.

15. THE TWO LEAF ARTIFACTS DO NOT CONTRADICT EACH OTHER. RESOLVED 2026-08-05, and the answer is a decomposition, not a winner.

`data/winrate-backtest.json` says the in-game leaf ranks at **50.99%** and is worse than a coin. `data/rollout-r1-explore-sweep.json` says the same leaf ranks at **69.84%** with a monotone reliability curve. The sweep flagged the conflict itself in `reading_against_the_leaf_calibration` and refused to treat "explore=1.0 spent the signal" as established while a second measurement disagreed. **It was right to refuse, and both artifacts are correct.** They score the same function on positions of very different difficulty, and the gap decomposes cleanly.

The sweep named two differences — rollout budget and horizon — and **those are the two that do not matter.** There are six, and the three that carry the gap are position, corpus and sheet, in that order.

`engine/leaf_position_contrast.js` holds five of the six fixed at a time: one leaf (`rolloutWinProb` , `explore=1.0`, `n=40`, `horizon 20`), one frozen release (**6boe4117d964**), the same seeds, and both accuracy definitions on every arm. `data/leaf-position-contrast.json` , with `data/leaf-position-contrast-rows-6b0e4117d964.jsonl` beside it so any cut can be re-derived without re-running the rollouts.

arm	corpus	position	sheet	n	maj. class	accuracy	Brier vs coin (paired)	ECE	MCE	curve slope
D = the sweep	open-sheet bo3	mid-game	yes	9,201 pos / 2,500 g	52.5%	69.83% [68.6, 71.1]	-0.0440 [-0.0513, -0.0360]	0.0925	0.162	0.703
C	open-sheet bo3	mid-game	no	9,201	52.5%	68.73% [67.5, 69.9]	-0.0401 [-0.0470, -0.0329]	0.0939	0.146	0.693
B	open-sheet bo3	turn o	yes	2,500	52.4%	58.20% [56.4, 60.2]	+0.0101 [0.0020, 0.0182]	0.1120	0.332	0.402
A	open-sheet bo3	turn o	no	2,500	52.4%	55.92% [54.0, 57.8]	+0.0166 [0.0088, 0.0243]	0.1284	0.351	0.331
E = the backtest	closed ladder	turn o	no	1,499	52.2%	51.17% [48.6, 53.6]	+0.0456 [0.0344, 0.0567]	0.1793	0.458	0.068

Intervals are game-clustered bootstraps, because 9,201 mid-game positions come from 2,500 games and an unclustered interval on them is too narrow by about $\sqrt{3.7}$.

The decomposition telescopes exactly. **69.83 – 51.17 = 18.66 points:**

term	contrast	points	how measured
POSITION	A → C	+12.81	mid-game vs turn o, sheet off, same 2,500 games
CORPUS	E → A	+4.75	closed ladder vs open-sheet bo3, turn o, sheet off, same config
SHEET	C → D	+1.10 [0.31, 1.88]	paired McNemar, same 9,201 boards from two walks

C and D come from two passes of `joint_rows.build` over the same games, the second with `Board.prototype.setSheet` disabled — so suppressing the sheet changes what the leaf KNOWS and must not change which boards are scored. **The original run asserted that and it PASSED:** all 9,201 positions agree across the two walks on gid, turn, label, `aliveDiff` and the continuous HP witness, and the run aborts rather than report a pairing it did not check. The artifact on disk is a re-cut of that run's rows, so its `pairing_check` says the result is carried rather than re-performed — a process that did not do the check does not get to say PASSED.

The sheet at turn 0 is worth **+2.28** [0.57, 3.99] (B – A, paired), so taking the other path through the square gives position +11.63 instead of +12.81. Either way position is two-thirds of it and the sheet is the smallest of the three.

Three independent things say the config is not the explanation. Arm E re-runs the backtest's condition at the SWEEP's budget and horizon (n=40, h=20) and lands on 51.17% / Brier +0.0456 / ECE 0.1793 against the published 51.66% / +0.0466 / 0.1827 — inside E's own split-half floor of 1.54 points. The sweep re-ran itself at h=60 and got 69.86% against 69.84%. And §1 already recorded that 40 and 200 rollouts give the same turn-0 answer. **The horizon and the budget are settled: they move nothing.**

Two independent routes reach the same turn-0 number, which is why I believe the decomposition. Cutting the sweep's own committed dump down to `turn ≤ 1 AND aliveDiff == 0 AND |hpDiff| < 0.02` — its nearest thing to a preview board, sheets on — gives **55.70%** on n=237. Arm B measures a real turn-0 board on the same corpus with sheets on and gives **58.20%** on n=2,500. The subset's split-half spread runs 0.47 to 21.47 points across ten random by-game cuts (median ≈ 4.2), so those two agree.

THE HEADLINE 50.99% IS THE UNDERPOWERED READ, and the better number is not better news. It is the held-out fifth at n=200. Re-cut from `data/winrate-backtest-rows.jsonl`, the same leaf at n=40 over the **full** 6,886-game clean corpus ranks at **51.66%** (51.80% on 6,570 decisive calls) — real by p, and its majority class is 51.25%, so its edge over *always say p1* is **0.41 points against a median split-half floor of 0.75**. LESSONS §9: an effect smaller than the noise floor is not an effect. **On the closed-sheet ladder at turn 0 the leaf does not beat the majority class.** "Cannot rank at all" was reported off the wrong n and happens to survive the correction.

Now the answer to the three options, plainly.

- **(a) "fine mid-game, broken at turn 0" — the largest term, and "fine" is too kind.** Mid-game the leaf is genuinely not broken: it beats a coin on Brier by 0.044 [0.036, 0.051], its curve is monotone with slope 0.703, and on boards where the material baseline has *collapsed to the majority class* (aliveDiff 0, |hpDiff| < 0.02, n=411) it scores 62.29% against material's 51.09% — **+11.19 [5.05, 17.34] over counting**. It is reading real non-material structure. But it still puts **31.4%** of positions in the two extreme bins, and its top bin predicts 97% and wins 86%. A slope of 0.70 is not calibration; it is a leaf that ranks well and lies about how sure it is.
- **(b) "broken everywhere, the sweep measures something easier" — right that it is easier, wrong that it is only easier.** The +11.19 over a collapsed material baseline is not an artefact of easy positions. The sweep is not measuring material with extra steps.
- **(c) and there is a term nobody named: the CORPUS, +4.75 points — bigger than the sheet channel at either position.** The open-sheet corpus is `fit_policy.loadCorpus()`, which on its first 2,500 games is **99.9% our own** `gen9championsvgc2026regmbbo3` **scrape**, not the OTS archive as the generator's own comment implies. Its pool is lower-rated (median 1,174 against the ladder held-out fifth's 1,266) and it plays under **forced** open sheets, so both humans had full information and the outcome may be more determined by the matchup. Turn counts and forfeit rates are the same in both. **Neither mechanism is tested here** — the term is measured, its cause is not, and it is not a sampling artefact of the held-out slice, because the full-corpus backtest agrees with E.

DISCRIMINATION AND CALIBRATION FAIL SEPARATELY AND THE SPLIT WIDENS AS INFORMATION IS REMOVED. Arm A ranks at 55.92% against a 52.4% majority — its interval's lower bound is 54.0, clear of both the majority class and its 2.24-point split-half floor — and its Brier is still **worse than a coin**, with an MCE of 0.351. So a turn-0 leaf can carry real ranking signal and still be a liar about its confidence, which is the failure mode that matters to an argmax. By arm E even the ranking is gone and only the confidence is left.

A SIGN FLIP WORTH A RE-RUN, EXPLICITLY NOT ESTABLISHED. Exploration helps mid-game and may hurt at turn 0. Paired on the backtest's own 6,886 turn-0 ladder games at horizon 60, the **greedy** playout ranks at 53.09% against explore=1.0's 51.66% — **+1.44 [0.10, 2.77]** — while paired on the sweep's 9,201 mid-game boards explore=1.0 wins by **+3.20 [2.24, 4.15]**. The turn-0 lower bound is 0.10 against a median split-half floor of 0.75, so it is **inside its own noise floor and is not a result**; and greedy's Brier there is *worse* (0.3240 against 0.2966), so the two playouts differ in which failure they have rather than in quality. The two arms are also not the same code path (`battleInit + chooseAction` against `rolloutWinProb`). It needs `mew.js --miltank-explore`, which §3 already filed to SEARCH, and it needs the position held fixed.

WHAT THIS MEANS FOR PORYZ, since that is what the question was for. `docs/PORYZ-spec.md`'s representation is per-Pokémon HP fraction, status, every stat stage, revealed item and ability, and a threat matrix — and it is the leaf of $EV(a) = \sum P(\text{reply}) \times V(\text{board after})$. **Every one of those inputs is constant across games or absent at turn 0:** all eight bodies are at 1.0 HP, no status, no stages, and the closed-sheet ladder has revealed no items. The only feature that survives to turn 0 is the threat matrix over the brought four a side. So **PORYZ cannot move the turn-0 number, by construction of its own feature list** — if the target was "the leaf that reads 100% and loses", that leaf is the PREVIEW one and this spec is not aimed at it.

Aimed at what PORYZ-spec's engineering section actually says — making the mid-game EV sum affordable — it is well aimed, and this run hands it a bar measured on the same positions rather than quoted from another sample: **69.83% accuracy, Brier 0.2060, ECE 0.0925, slope 0.703 on 9,201 positions at release 6boe4117d964**, with the rows on disk. PORYZ's premise sentence, "the whole learned value function is worth 3.4 points over counting", is about PORY2. The rollout leaf is worth **+4.58 [3.47, 5.68]** over the same graded material baseline mid-game and **+11.19 [5.05, 17.34]** where material has nothing to say. That is the incumbent PORYZ has to beat, and it is a harder incumbent than the spec assumed. This is a measurement, not a build decision; the decision is SEARCH's.

Filed, not fixed.

- `engine/rollout_r1.js:436` **puts a prose note key inside** `source_digests`. `engine/provenance.js:648` calls `digestOf()` on every key in that map, so a key that is not a readable path marks the whole artifact `unverifiable` — which is why `data/rollout-r1-explore-sweep.json` cannot be digest-verified. This file made the same mistake and moved the prose to a sibling key; the artifact went from `stale?` to `ok`. SEARCH's file.
- `engine/rollout_r1.js:26-29` **'s corpus comment is misleading about what it samples.** `loadCorpus()` reads `bo3`, `OTS` and `ladder` in that order, and the first 2,500 games — the whole `R1` and `sweep` sample — are 2,497 `bo3` and 3 `smogtours`. Every `R1` number ever published is a **bo3 open-sheet** number, and §15 measures that this corpus is worth 4.75 accuracy points at turn 0. The published figures are not wrong; what they are *about* is narrower than the comment says.
- `data/censoring-value.json` **and** `data/click-censoring-census.json` **trip the provenance ratchet** — their generator ships without recording what content it read. Another division's files, written this session. Reported, not touched.

Re-cutting this artifact costs seconds, not half an hour:

```
RECUT=data/leaf-position-contrast-rows-6b0e4117d964.jsonl node engine/leaf_position_contrast.js
```

It opens the release named in the filename rather than the newest, refuses a dump that cannot name its engine, and never rewrites the dump it read. Verified: the re-cut reproduces every figure above bit-for-bit and leaves the row file byte-identical.

§16 — censoring-value.json is UNSAFE, and re-running it is not a repeat

ANSWERED IN §17, 2026-08-05 — and by none of the three options below. *The confound was measured instead of argued: all 58 feature columns are identical across the engine bundles on all 1,751,688 corpus rows, so the fitting environment and the playing environment are the same FUNCTION here. Both artifacts were re-run against the live tree and both are ok . The section below is kept as what was true before that was measured; do not read its three options as open.*

2026-08-05. provenance.js flags it: medicham2-browser.js was e2bcff0db96f when it was measured and is 80fe43fba1a9 now, because WIRES 114–116 landed underneath it. The flag is correct and the artifact should not be quoted.

Two things had to be fixed before it could even be re-run, and both are worth more than the number.

The comparison baseline lived in a session scratchpad. The run compares the pre-censoring incumbent against the post-censoring fit, and the incumbent existed only as a copy in a temp directory that gets cleaned. A published figure whose input is in %TEMP% is not reproducible by anyone, ourselves included, one cleanup later. It is now data/policy-weights-pre-censoring.json , sha12 01bc43936324 — the digest the artifact itself records, verified to match.

The artifact was invisible to provenance despite recording more than most files that pass. It stamped source_digests_before and source_digests_after ; provenance.js reads source_digests and nothing else, so it fell to "rests on mtime alone" while carrying better evidence than the files around it. Recording something correctly under a name the checker cannot see has the same outcome as not recording it. The generator now writes the canonical key too — and the moment it did, the artifact stopped being ok and became UNSAFE , which is the whole point.

THE RE-RUN IS BLOCKED ON A JUDGEMENT, NOT ON COMPUTE. Both weight vectors were fitted under the pre-WIRE-114 engine. Scoring them through the current one breaks the rule in CLAUDE.md that the fitting environment and the playing environment must match, and it would measure *the censoring change plus three wires* as one quantity. The options, none free:

- **Refit both vectors under the current engine, then re-measure.** Correct, and the expensive one.
- **Re-measure through a release frozen at e2bcff0db96f .** Reproduces the original honestly, but the artifact deliberately reads the live tree — no_engine_release says freezing it would measure the thing being tested — so this changes the design of the measurement.
- **Leave it UNSAFE until the next refit lands anyway,** and do not quote it. Cheapest, and the status quo, but only honest while nothing downstream depends on it.

Not chosen here. engine/censoring_value.js refuses to run without WEIGHTS_OLD and now points at the preserved baseline and at this section, so whoever picks it up is choosing rather than guessing.

§17 — THE CONFOUND WAS MEASURED AND IT IS EMPTY. Both artifacts re-run, both ok . 2026-08-05.

None of the three options in §16 was taken, and the reason is a measurement rather than an argument. The blocking question — *"the vectors were fitted under one engine and would be scored through another"* — is a claim about the FEATURE FUNCTION, and a feature function is a function from a board to a number. Two versions of it are the same function if they agree on every board. So they were run against each other on every board the fit actually uses.

Result: all 58 feature columns are hash-identical across the three engine bundles, over 1,751,688 candidate feature vectors from all 9,230 clean open-sheet games.

bundle	medicham2-browser.js	data/tags.json	what read it	58 column hashes
release 09acd3b404ef	e2bcff0db96f	c0bb781f47a8	censoring-value.json	identical
release 032b4a2979dd	80fe43fba1a9	c0bb781f47a8	click-censoring-census.json	identical
live	0cb911437fed	73c81e6421b8	the re-runs below	identical

The three bundles were loaded from the frozen releases and registered under the live module paths, so `board.js`, `fit_policy.js` and `click_match.js` are the same bytes in every arm and only the simulator and the tag dex move. `engine/quality.js` is deliberately NOT swapped — a snapshot copy of it resolves the store inside the release directory and the walk would have had no rows to disagree about.

A null result from an instrument that cannot see is worth nothing, so the instrument was shown seeing. Under a Psychic Terrain with a Levitate body, the two frozen engines return `0` — priority refused — and the live one returns `Infinity`. The harness reads the same call the feature code reads, so a difference of that kind would have moved a column.

Why the change is real in the simulator and invisible in the features: across the whole corpus `board.js` makes **173,478** guarded calls to `priorityRefusedAbove`, of which **424** are under a Psychic Terrain, and in **0** of them is every live defender airborne. WIRE 117 can only change an answer when no grounded body is left to hold the bar up.

Both artifacts were then re-run against the live tree, and both reproduce. The corpus had grown 8,942 → 9,022 → **9,230** clean open-sheet games in between, so this is a fresh measurement on a superset rather than a replay — which makes the agreement evidence rather than tautology:

held-out class	published 3.42.0 (n=47,195, 1,809 games)	re-run (n=48,274, 1,851 games)
COERCED P(the coerced action), lower is better	-0.002614 [-0.003663, -0.001637]	-0.002613 [-0.003650, -0.001672]
PARTIAL mass on the candidate set	+0.000109 [-0.000286, +0.000491]	+0.000122 [-0.000261, +0.000514]
PARTIAL log-likelihood of the set	-0.002646 [-0.004037, -0.001377]	-0.002662 [-0.004002, -0.001368]
CONTROL, CLEAN log-likelihood	+0.000447 [0.000142, 0.000743]	+0.000485 [0.000189, 0.000777]
CONTROL, CLEAN top-1	+0.002 [-0.094, 0.098]	-0.008 [-0.107, 0.085]

Every verdict in §14 stands, including the negative one: the redirection correction still buys nothing measurable, and **every effect is still smaller than its own class's split-half floor** (COERCED 0.002613 against 0.011909; CLEAN logL 0.000485 against 0.004820). They resolve because the comparison is paired per decision, and that sentence must keep travelling with the numbers.

The census moved with the corpus and its shares did not: **249,404 actions over 9,230 games — CLEAN 94.9111%, PARTIAL 1.3344% (3,328), COERCED 0.5545% (1,383: Encore 1,152, |drag| 231)**, against 94.8916 / 1.3467 / 0.5509 at 9,022 games. The classifier still scores against the raw protocol at **encore recall 99.69% precision 96.31%, drag 96.74% / 96.74%** on the 67.23% of games that have a raw log.

node engine/provenance.js --strict **exited 0 at that point: 0 UNSAFE, 1 declared VOID** (exploitability.json), **57 ok**. Both files carry source_digests over the tree they were computed on, so a next engine move flags them again by CONTENT rather than by mtime — **and one did, forty minutes later. See §17b, which is the more important half of this section.**

What this does NOT license. It says the four wires moved no feature on THIS corpus — it does not say the engine did not change, and it is not a general permit to score old weights through a new simulator. The next engine move gets the same treatment: run the columns, then decide.

The harness is engine/feature_engine_contrast.js **and it is in the repository, not in a session scratchpad — which is §16's own lesson applied to §17's evidence.** It writes data/feature-engine-contrast.json with source_digests , and it costs about four minutes per bundle over the whole corpus:

```
SHOWDOWN_PATH=... BUNDLES=live,09acd3b404ef,032b4a2979dd node engine/feature_engine_contrast.js
```

Each bundle runs in its own child process, because a module-cache swap cannot be undone in one. Two properties are worth more than the number it prints:

- **It refuses to report agreement unless its positive control disagreed.** BUNDLES=live, live returns *NOT A RESULT* — the positive control did not separate the bundles, verified before this was believed. A harness that silently loaded the same bytes twice would otherwise publish a confident "identical", which is the exact shape of every failure in this project's history.
- **It is not** engine/feature_fixture.js **and does not replace it.** The fixture hashes ~50 frozen boards so a weight file can carry the hashes, and its own header states the limit: a guard only guards what it exercises. This runs the same question over every board the fit actually uses, so a branch no fixture board stands on cannot hide in it. Both were green here, which is the first time they have been asked the same question on the same day.

§17b — AND THEN THE TREE MOVED AGAIN, AND THIS TIME THREE COLUMNS MOVED WITH IT. A REFIT IS OWED.

The instrument built in §17 found a real feature change forty minutes after it was written, and engine/feature_fixture.js --check — the guard status.js **prints the refit edge from — is BLIND to it.** That is the finding of this session, and it outranks everything above.

Between 15:40 and 15:44 on 2026-08-05, while this division was measuring, three files moved:

file	was	is	what it did
engine/fit_policy.js	45f545425420	caeeec21c560	loadCorpus() went 9,230 → 6,055 clean open-sheet games
engine/medicham2-browser.js	0cb911437fed	82bed8cdcf6b	—
engine/board.js	54e3d2ca9f85	5bdaa3923958	the feature file itself

Re-run with the sample pinned — **1,136,845 candidate vectors over the same 6,055 games, identical row_key_hash in all three arms** — the verdict is no longer IDENTICAL:

MOVED — deadNoLastMove , movesFirst , diesBeforeMoving **differ on identical rows. This is a REFIT, not a restamp.**

Both frozen bundles (e2bcff0db96f , 80fe43fba1a9) agree with each other and disagree with the live tree in the same three columns, which is what a single new change looks like — ****and it is: CHANGELOG 3.49.0, "There were two implementations of who moves first. One is deleted, and the survivor is dynamic."**** Speed

order is now re-sorted mid-turn, so `movesFirst` and everything downstream of it answers a different question than the weights were fitted against. The columns name the change without anyone having to guess, which is what a per-column hash is for.

**** `node engine/feature_fixture.js --check data/policy-weights.json` says "feature semantics OK — agrees with `board.js` on every fixture board" on that same tree. Both instruments are working; they are answering the question on different boards, and the ~50 frozen fixture boards do not stand on the branch that moved. The fixture's own header says a guard only guards what it exercises — this is the first time that limit has been shown with a number rather than stated. `status.js` prints `refit edge: CLEAN` from that check, so the refit edge is currently reported clean and is not.**** Two consequences, in order:

- 1. **A refit is owed on the 15:43–15:44 change** — three of the 58 columns changed meaning under weights fitted against the old ones. That is not WIRES 114–117; those were measured empty above.
- 2. **The refit edge needs both instruments.** The fixture is what a weight file can CARRY, and it should stay; the corpus contrast is what can DETECT. Wiring `feature-engine-contrast.json` into `status.js` beside `feature_fixture --check` is the obvious next move, and it is deliberately not done in this pass — a status line added at the end of a session that watched three files move is a line nobody has watched behave.

`data/click-censoring-census.json` and `data/censoring-value.json` are therefore UNSAFE again, now through `engine/fit_policy.js` rather than through the simulator, together with `data/partial-label-em.json`, which is the same cause and was not touched here. `node engine/provenance.js --strict` **exits 1 with 3 UNSAFE**. That is stated, not filed: the re-runs in §17 were valid photographs of the tree at 14:26–14:40 and they say so in their own digests; the tree they photographed no longer exists.

They were not re-run a third time, deliberately. The corpus definition changed by a third (9,230 → 6,055 open-sheet games) inside the same twenty minutes, so a third run would publish a different population under the same headline, attributable to neither the engine nor the censoring change. Re-run both against a still tree — the loader digest is in every artifact — and the numbers in §17 are the ones to compare against.

A measurement cannot be taken while the lens is being changed, and `engine_release.js` does not cover this case. A release freezes 23 files; it does not freeze `engine/fit_policy.js`, and it cannot freeze the store. That is why `feature_engine_contrast.js` pins its sample by game id and refuses when one goes missing: the first version of it reported **all 58 columns moved** purely because the corpus shrank between two children, which is a REFIT verdict manufactured out of somebody else's edit.

§17a — the `board.js` partial-body over-refusal is worth 0 rows, and here is the number ENGINE filed it rather than fixing it: `engine/board.js:2565` and `engine/position_features.js:231` map their priority defenders to `{ability, fainted}`, so `isGrounded()` sees no type list and no item and a Flying-type foe is still over-refused **in the feature vector**. Widening that signature moves the feature vector, which is a refit, which is why it came here. Measured on the fit's own decisions over all 9,230 games, rebuilding every defender twice — once the way `board.js` does it, once with the types and item the board already holds:

	n	of
candidate feature vectors	1,751,688	—
with a priority move	332,030	19.0% of candidates
aimed at a body, i.e. reaching <code>board.js:2560</code> 's guard	135,552	40.8% of those
under a Psychic Terrain	362	0.27% of guarded priority candidates
where a complete body changes the answer	0	—

The artifact on disk carries the same measurement over the post-15:40 corpus (6,055 games, 1,136,845 vectors, 220,932 with priority, 91,240 reaching the guard, **273** under a Psychic Terrain, **0** changed, upper bound 5). Two corpora a third apart give the same answer, which is the strongest thing that can be said about it without more Psychic Terrain in the metagame.

The only five rows in the entire corpus where types and item flip the bar are `protect` ×4 and `ragepowder` ×1 — **self-targeted moves, which** `board.js` **never routes through** `priorityRefusedAbove` **at all**, because the branch is guarded on `cand.targetMon`. Counting them as exposure would have overstated it by five rows out of 1.75 million; both counts are recorded here so the guard is visible rather than assumed.

So: NOT WORTH A REFIT, and the exposure is 0 rows in 1,751,688 (upper bound 5, of which 0 are reachable). Two things keep it from being closed. `fails.groundedBodyIncomplete` fires on **100% of 173,478** calls — every single feature-path call is made with a body that cannot answer — so the defect is total and only its consequence is nil; and the consequence is a property of THIS corpus, where 0.27% of guarded priority candidates stand on a Psychic Terrain. A metagame that pairs Psychic Surge with Flying bodies moves that number without anything in the code changing. The right time to widen the signature is the next refit, when the feature vector is moving anyway and the change is free. `engine/position_features.js` 's copy is a separate call site and is NOT measured here.

Reading a run

```
node engine/sprt.js <file>
```

Cat the shards together first. **Never read an interim SPRT** — 66.7% became 44%, 57.7% became 50%. The bound exists precisely so you do not have to look. SPRT is valid under continuous monitoring because its boundaries were derived for it; a Wilson interval read repeatedly is not the same thing and does not inherit that property.

The unit is the **decisive pair**, not the game. In a paired run a 1-1 split means the team decided it, not the policy.

Reading a stamp

```
node engine/run_stamp.js --show          data/rollout-r3.json
node engine/run_stamp.js --reconstruct data/rollout-cost.json
```

Every gate artifact has a `<name>.meta.json` beside it saying which configuration produced it. `status.js` prints the headline under the gate line, so the absence of a stamp is on the same screen as the number — R1's +2.91 was quoted for a day against a dump that could not say which of two runs four accuracy points apart it was, and nothing was hidden then either. The fact simply lived in a file nobody opened.

Three things to check before quoting any of it:

- `reconstructed: true` means **inferred from a commit, not observed**. Read `confidence`, which publishes the gap in seconds between the artifact's own timestamp and the commit that carried it.
- `git.dirty: true` means the commit id does not describe what ran. Trust `source_digests`.
- `source_digests` hashes **worktree bytes**; `git.blobs` names git objects. On Windows those differ by line-ending translation — `data/engine-data.js` does — so never compare one to the other.

`writeStamp()` is the only mode worth trusting, because only the run knows its own settings. `reconstruct()` exists for the artifacts that predate it and labels itself on every line.

18. THE PORYGON2 SEPARATION GATE — PRIORITIES #23. PASS, and the interesting number is the one that is not in the verdict. 2026-08-06

engine/porygon2_separation_gate.py → data/porygon2-separation-gate.json . The MILTANK leaf redesign (#24) is buildable. PORYGON2 does not collapse a subtree to one number.

39,843 same-game position pairs two turns apart, across 6,328 clean HUMAN ladder games, every interval bootstrapped with the GAME as the cluster. Thresholds were written to disk at **05:59:44Z**, the run wrote at **06:56:06Z**, and `--run` refuses to start unless the declaration on disk matches the block in the generator character for character.

	measured	declared bar			
T1 separation median \	Δ score\	over 2 turns	0.1628 [0.1600, 0.1653]	≥ 0.02	PASS
T2 locality same-game 0.1985 vs unrelated 0.2801; D	+0.0815 [0.0786, 0.0845]	CI lower > 0.0043	PASS		
T2 locality ratio R = same / unrelated	0.709 [0.700, 0.718]	≤ 0.75	PASS		
T3 direction agrees with the material sign	85.58% [85.16, 85.98]	CI lower > 50, point ≥ 60	PASS		
T3 secondary, moves toward the eventual winner	61.59% [61.12, 62.07]	reported, not gated			

All eight PORYGON2 arms pass — 17 and 19 features, plain and weighted, k=50 and k=200 — with R between 0.684 and 0.739. The verdict is read off **17f weighted k=50**, which is what docs/MODELS.md headlines.

THE NEGATIVE CONTROLS DID THEIR JOB, AND THE SECOND ONE IS THE ONE THAT MATTERS. A constant 0.5 leaf fails all three (median 0, R undefined, direction 0%). That was the required control and it is the weaker one. A **uniform-random** leaf **PASSES T1 with a median of 0.2924 — nearly twice PORYGON2's separation** — and fails T2 (R = 0.995 [0.985, 1.004], D CI [−0.0012, +0.0049] straddling zero) and T3 (50.28% [49.72, 50.87]). So separation alone cannot tell a value function from noise, which is precisely why T2 was written as the deciding test. A gate proved only against a constant would have been passed by static.

AND THE FINDING THE VERDICT DOES NOT CONTAIN. A bare material count — 0.5 + 0.15·alive_diff , the same rule porygon2.py scores itself against — was run through the identical pipeline as a BASELINE rather than a control. At a two-turn gap **it passes the gate too**: R = 0.703 [0.692, 0.715], statistically indistinguishable from PORYGON2's 0.709. Read alone, that says the 17 features buy no locality at all.

It is not read alone, because the addendum below settles it. **At the ONE-turn gap the search actually operates at, the material count goes flat: its median $|\Delta|$ is 0.000 and it returns the identical number on 58% of adjacent positions**, while PORYGON2 moves on 99.3% of them with a median of 0.1154 and its locality gets *better*, R = 0.5464 [0.5392, 0.5534]. Every branch a material leaf cannot separate is a branch the argmax decides by tie-break. That is the case for #24, and it is a different case from the one the headline makes.

Two comparisons in that block that look like findings and are not, stated so nobody quotes them:

- the material baseline's *toward-the-eventual-winner* rate (64.77%) is **higher** than PORYGON2's (61.59%) — but its score moves on only 21,975 of 39,843 pairs, i.e. only where a Pokemon actually fainted. It is scoring the easy subset. The two rates are computed on different populations and are not comparable.
- the gate produces properly-intervalled accuracies for free: **17f weighted k=50 at 63.11% [62.32, 63.81]** against the material sign's 61.02% [60.06, 61.96] on the same 52,501 positions. These are **separate** game-clustered intervals, **not a paired test**. docs/MODELS.md 's 63.59% is still marked **NOT**

MEASURED and this **supersedes nothing** — a paired difference with a split-half floor is what would close it, and nobody has run one.

WHAT WAS FROZEN, AND WHAT COULD NOT BE. The gate is stamped to engine release `4c73f9cafa4b` and that stamp is honest about its own limits: **none of PORYGON2's sources are in the frozen set** — not `engine/porygon2.py`, not `data/porygon2-species.json`, not either corpus. PORYGON2 is a Python model and `REL.require` is a JavaScript shim, so it cannot be loaded through a release at all. What the release *did* supply, through `REL.require`, is the thing that decides the population: the frozen `engine/quality.js` + `data/quality-filter.json`. For the rest the generator takes its own photograph — sources copied into a private tree and imported from the copy, live originals re-digested afterwards (none moved) — and the two append-only stores are pinned by the **clean id set** (7,992 ids, sha256 `4ccc0afc...`) rather than by a whole-file digest, because the collector appends hourly and a file digest would void any run longer than an hour.

A DEFECT FOUND ON THE WAY, AND IT IS THIS REPOSITORY'S SIGNATURE SHAPE. The first artifact carried `"R_same_over_unrelated": NaN` — Python's `json.dump` writes a bare `NaN`, which every Python reader accepts and which **is not valid JSON**. `JSON.parse` throws on it. The effect was that `engine/provenance.js` **could not read one field of the file and reported it** `ok`: a clean bill of health issued over a document it had never parsed, including the `void` flag that exists precisely so a generator can condemn its own run. Both dumps now pass `allow_nan=False`, so it raises instead of shipping. Worth a sweep: any Python generator here can emit this, and the artifact still looks fine from Python.

Three smaller things the gate needed and now does:

- it writes `corpus.clean_games` and `corpus.population_ceiling` **spelled the way** `provenance.js` **reads them**. Its prose `population` block was invisible to `declaredGamesFrom()`, which is the §5e state where an artifact "records no game count".
- the declaration timestamp survives every re-run. `--run` overwrites the file, so reading `generated` would report the last run as the moment the thresholds were fixed — drifting later than the numbers, every time.
- a `--run` re-run used to silently delete the `--addendum` block. It is carried forward now, each block keeping its own timestamp and digests.

Disclosed rather than omitted: a 150-game smoke run of this pipeline executed at 06:02Z, after the declaration and before the headline sample, to find bugs. Its numbers were seen first. No threshold changed — the equality check enforces that — but the smoke run's R landed at 0.735 against a 0.75 bar, close enough that saying nothing about it would be the omission this division exists to prevent.

What this gate does NOT establish, and #24 should not be read as having it:

- it says the leaf **separates**, not that swapping it in **wins**. The unit that answers that is the decisive pair, and it needs an SPRT against the incumbent playoff.
- the pairs are consecutive positions from *real games*, not **sibling branches from one node**. Siblings differ by one action from an identical board and are more alike than anything measured here. The lag-1 addendum is the closest available proxy and it is a proxy.
- T3's ground truth is `alive_diff + hp_total_diff`, which are two of PORYGON2's own inputs (`alive_diff` carries a learned weight of 5.12 against a mean of 1.0). It asks whether the model respects its strongest features. A k-NN guarantees no such thing, so it is not vacuous — but it is not independent, which is why the outcome-anchored secondary is reported beside it.
- **no split-half was run**. The noise floor here is built into the design instead: T2's unrelated-pair arm is the floor for the effect claimed, and every interval is game-clustered. The estimator is deterministic given the game set, so a split-half would re-measure what the bootstrap already reports. The one stochastic input

— how the unrelated partner is drawn — was checked by a second mechanism: the any-turn control gives $R \approx 0.74$ against the turn-matched 0.709, so the conclusion does not depend on the draw.

19. THE STRONG-PLAYER BASELINE — the cutoff gradient exists and is free; §1.3's "real humans" column cannot be compared to it; and "flat in rating" is NOT MEASURED. 2026-08-06

`data/strong-player-baseline.json`, written by `build/strong_player_baseline.js` (one process, ~2 minutes). Built for task #46 out of `data/smogon-stats/` and this repository's own stores. **It reads no simulator, no leaf, no feature layer and no policy weights**, so it is not invalidated by an engine release or by a MAG refit and does not need re-running when either happens. It is invalidated by a new Smogon month or by a change to `data/quality-filter.json`.

The generator exists because of §19e. The claim this section retracts — *"measured move quality is close to flat in rating"* — has been quoted in a fitted model's own caveat block for weeks with no generator behind it. Shipping an artifact with the same defect would have been the same mistake in a new file. It lives in `build/` rather than `engine/` for a dated reason: it was written on 2026-08-06 while an ENGINE agent was rewriting the simulator, and this division does not add files to another agent's directory mid-flight. If that reason expires, `engine/` is the better home.

The question it was built for is Will's, 2026-08-06, reading `docs/ROADMAP.md` §1.3: *"outright failed' could be incompetence or a high level play and we dont know the difference."*

19a. What the 1630 weighting can and cannot support — stated before it is used

It CAN support what strong players BRING and RUN. Species usage at four skill weightings, and ability / item / spread / move frequencies within a species.

It CANNOT support what strong players CLICK. The files are team-composition aggregates. There is no turn, no board, no opponent and no click in them. All three of §1.3's metrics — *moves that outright failed*, *moves that hit an immune target*, *moves that were super effective* — are per-turn rates and **have no Smogon counterpart at any cutoff**. A 1630 column added to that table would look comparable and would not be, which is worse than a missing column.

"Cutoffs are weightings, not subsets" is now MEASURED rather than quoted. Across both months and both formats, every species' `Raw count` is identical at all four cutoffs — **3,117 of 3,117 species-cutoff pairs**, and 310 of 310 rows of the usage table's `Raw` column. Only `Avg. weight` moves. So no cutoff describes a *set of players*; each describes the same battles seen through a different lens.

And no rating number is mapped onto a cutoff number anywhere in this work. Nothing in this repository or in the Smogon files establishes that 1630 is the same ruler as the Showdown `|player|` rating field. The corpus is located on the cutoff axis by **composition**, which is scale-free.

19b. The gradient exists, and it was free

Four cutoffs (0 / 1500 / 1630 / 1760) × two months (2026-06, 2026-07) × two formats (Bo1, Bo3) are already on disk: 16 usage files and 16 moveset files. Nothing was collected.

Effective sample size is `Raw count × Avg. weight` per species per cutoff. Smogon weights lie in $[0,1]$, so $\Sigma w^2 \leq \Sigma w$ and the true effective sample $(\Sigma w)^2 / \Sigma w^2$ is **at least** Σw — the intervals below are therefore too WIDE, not too narrow. The offsetting hazard is stated and not corrected for: the independent unit is a PLAYER, one strong player contributes many battles, and the files cannot measure that.

The 1760 column costs almost everything. Effective team slots, 2026-07 Bo1:

cutoff	avg weight/team	effective team slots	share of raw
0	1.000	3,529,372	100%

cutoff	avg weight/team	effective team slots	share of raw
1500	0.512	1,807,038	51.2%
1630	0.068	239,997	6.8%
1760	0.002	7,059	0.2%

The noise floor is the same cutoff across two months, which is an *upper* bound because it contains real metagame drift as well as sampling. Total absolute species-usage difference, summed over 310 ranked species, 2026-07:

contrast	L1 (points)	vs the cutoff-0 month floor of 140.9
cutoff 0 vs 1500	69.4	inside it — 1500 is not distinguishable from the whole ladder
cutoff 0 vs 1630	151.6	above
cutoff 0 vs 1760	195.0	above
cutoff 1630 vs 1760	70.6	inside

83 of the 104 species at ≥1% base usage have a 0→1760 usage change whose 95% interval excludes zero, and the large ones are monotone across all four cutoffs. Charizard-Mega-Y 15.69% → 18.75% → 23.46% → **30.58%** (Δ +14.89 [13.81, 15.96]); Kingambit 23.40 → 36.79 (+13.39 [12.27, 14.52]); Sableye 6.52 → 3.36 (−3.17 [−3.59, −2.74]). (*Population: Smogon 2026-07* `gen9championsvgc2026regmb` , 1,764,686 battles, *reweighted*.)

Within a species, the gradient is real for **moves and spreads**, marginal for **items**, and absent for **abilities**. Mean total-variation distance over the top 20 species by raw count, cutoff 0 vs 1760, computed over the **intersection** of the two listed key sets:

section	cutoff gradient	month noise at cutoff 0
moves	17.89	11.52
spreads	9.17	3.72
items	6.98	4.99
abilities	1.63	5.12

The abilities row is the honest negative, and it needs one species named: the 5.12 month-noise is dominated by **Sneasler**, whose Unburden/Poison Touch split really did move 15.3 points between June and July. Excluding it, abilities barely move with skill either — the format's abilities are near-locked at every cutoff.

A correction made mid-run, recorded because a first pass shipped it. A key absent from a Smogon moveset list is **not 0%** — the file lists the top few plus `other` , so an absent key is below that list's reporting floor. Treating it as zero manufactured an 18-point "gradient" on Venusaur/Energy Ball. Every distance here is over the intersection, with the unlisted mass reported separately.

19c. Where our corpora sit on that axis — and there are THREE of them, not one

They must never share a sentence with only one population named.

corpus	store	filter	rated slots	median	≥1400	≥1500
clean closed ladder	<code>games.ladder.jsonl</code>	clean, 8,047 games	11,852	1266	26.2%	14.4%
Bo3 open sheet	<code>games.bo3.jsonl</code>	none	14,539	1175	5.33%	0.72%
Bo1 open sheet	<code>games.ots.jsonl</code>	clean, 2,860 games	3,414	1087	0.23%	0.09%
unfiltered ladder	<code>games.ladder.jsonl</code>	none, 45,006 lines	83,668	1130	6.21%	3.09%

Two things fall out of that table.

The dispatch's figures are confirmed and they belong to the Bo3 store. 14,465 rated slots at p10 1043 / median 1175 / p90 1355 / max 1707, ≥ 1400 5.3%, ≥ 1500 0.7% is `data/games.bo3.json1`, which now holds 14,539 slots at exactly those quantiles. Same file, 74 slots of growth.

The clean filter moves the population a long way, and that is the population §1.3 benchmarks against. Unfiltered ladder median 1130 with 6.21% ≥ 1400 ; clean ladder median **1266** with **26.2%** ≥ 1400 . The bot and behavioural-bot rules remove a large low-and-flat block. So §1.3's "real humans" are not the median-1175 population — that is the corpus MAG was *fitted* on. Any sentence of the form "the ladder is median X" has to say which of the two it means.

On the cutoff axis, by composition (L1 between the corpus's team-preview species vector and each cutoff, mega formes collapsed to base on both sides):

corpus	vs cutoff 0	1500	1630	1760	own split-half floor
clean closed ladder, vs 2026-07	138.6	174.9	230.9	274.6	36.9 – 75.8 (median 53.4)
Bo1 open sheet, vs 2026-06	211.2	184.0	159.2	157.1	45.9 – 91.8 (median 67.3)

The closed ladder is nearest cutoff 0 and moves monotonically away from every higher one. Its 138.6 is about one month of drift at cutoff 0 (140.9) and its games run 2026-07-22 to 2026-08-06 — later than the newest published month — so cutoff 0 is a fit and 1630/1760 are not. **The Bo1 open-sheet corpus cannot be placed at all:** its four distances span 157.1–211.2 against its own split-half floor of 45.9–91.8, so the cutoff ordering is inside its noise. NOT MEASURED for that store, and the apparent preference for 1760 is not a finding.

19d. The Fake Out / Armor Tail case, answered as far as it can be

Farigiraf runs Armor Tail on **97.80%** of sets at cutoff 0 [97.77, 97.83] and **99.11%** at 1760 [98.59, 99.45] — a real +1.31-point gradient [0.89, 1.73] against a month noise of 0.11, and **completely useless for the question**, because the ability was already near-universal everywhere. Incineroar carries Fake Out on 98.89% of sets at cutoff 0 and 99.82% at 1760. Expected Fake Out carriers per team of six: 0.728 at cutoff 0 → 0.751 at 1760.

So the collision is **at least as available** at the top as at the bottom. The aggregate can say the two sides are both brought slightly more by strong players; it cannot say who clicked, because there is no turn in the file.

What this implies for task #44 part 1, which is owned elsewhere and NOT attempted here. The denominator of a failed-move rate is composition-confounded, and the split has to condition on it. Protection is the largest single source of a `|-fail|` line — a repeated Protect fails by rule — and it moves with the cutoff: expected protection carriers per team of six run **4.00** → **4.14** → **4.34** → **4.39** across the four cutoffs, and Detect alone runs 0.103 → 0.144 (+39% relative). A raw failed-move rate therefore rises with how much protection the population runs, independently of anybody playing better or worse.

19e. `fit_policy.js:1264` 's "flat in rating" — NOT MEASURED, not false

The claim: *"Open-sheet players also average ~185 rating points lower, though measured move quality is close to flat in rating."* `docs/DEFENSE.md` §1 gives the numbers — failed moves 2.59% under 1100 against 2.30% at 1400–1600; blocked actions 4.66% to 3.43%.

Neither figure has a generator in this repository. `engine/realism_report.js` counts the same protocol lines but **pools both players of a game and never bands by rating**, and no `data/*.json` carries a rating-banded rate. The claim has sat in a fitted model's own caveat block with nothing behind it that can be re-run. That is the P1 class this division already named for PORY's coefficients.

Recomputed here from the protocol, attributed to the **acting** side, on the clean closed ladder — 8,047 clean games / 7,040 raw logs matched / 14,078 player-slots / **171,801 moves**. Intervals are a game-clustered bootstrap, 400 resamples.

rating band	moves	failed % [95%]	immune % [95%]	super-effective % [95%]
<1100	21,098	2.218 [1.93, 2.57]	2.318 [2.01, 2.63]	20.84 [20.02, 21.59]
1100–1199	27,390	2.472 [2.08, 3.03]	2.234 [1.98, 2.47]	20.62 [19.97, 21.33]
1200–1299	22,699	2.291 [1.99, 2.59]	2.071 [1.83, 2.31]	20.62 [19.90, 21.32]
1300–1399	22,577	2.197 [1.95, 2.48]	2.210 [1.98, 2.46]	20.84 [20.16, 21.56]
1400–1599	24,669	2.165 [1.83, 2.53]	2.027 [1.81, 2.25]	21.24 [20.55, 21.91]
1600+	7,486	2.658 [1.93, 3.52]	2.151 [1.72, 2.57]	19.16 [17.95, 20.36]

The direction reproduces. <1100 against 1400–1599 on failed moves: **−0.054 points, 95% CI [−0.539, +0.403]**. Nothing here contradicts "close to flat".

But the design was powered for a 0.674-point difference at 80% power on a 2.2% base rate — a 31% RELATIVE change. Anything smaller was never detectable, so "*flat*" and "*an effect up to 30% of the base rate*" are the same observation in this corpus. Immunity, same two bands: −0.291 [−0.679, +0.057], MDE 0.517 on a 2.32% base. Super effective: +0.396 [−0.606, +1.461], MDE 1.527 on a 20.8% base. All three contrasts contain zero.

And the whole between-band spread is inside the within-band noise floor. Failed-move rate across all six bands spans 2.165% to 2.658% — 0.49 points. Cutting a *single* band eight ways produces spreads from **−0.489 to +0.806** points. The observed effect is the noise floor.

The binding constraint is not the rating range, which is what the dispatch expected. The clean closed ladder holds 32,155 moves at ≥1400, 17,551 at ≥1500 and 7,486 at ≥1600. The binding constraint is that the metric is a ~2% event: 25,000 moves is only ~530 failures, and separating two bands by a fifth of a point on that needs far more.

So the verdict is NOT MEASURED. The claim is not shown false and it should not be repeated as though it were established. `fit_policy.js:1264` and `docs/DEFENSE.md §1` should say *not measured at this power* — filed, not edited, because `engine/` is being rewritten in parallel.

What would settle it is not more games at this metric. Either a metric with a higher event rate (super-effective is 21%, so its 1.53-point MDE is **7.3% relative** against failed-move's 31% — four times better), or a paired design that holds the board fixed, which is what a click-level model gives and a rate does not.

19f. What §1.3 should say instead

The gap §1.3 reports between MAG and humans is **3.87 points** (6.34% against 2.47%). The entire measurable rating effect inside the human population is at most **0.67 points** and its point estimate is **0.05**. The gap is ~5.7× anything skill does to this metric within the corpus. **Closing it makes MAG resemble a human; it cannot make MAG resemble a strong human, because on this metric strong and weak humans are not separated at all.**

Five changes, in order:

- 1. Name the population on the same line as every figure.** The column is *clean closed-sheet ladder games, both players pooled, no rating condition*. It was 1,905 clean games when written; the same predicate now selects 8,047 clean games / 7,040 raw logs / 171,801 moves at median rating 1266 with 26.2% of rated slots ≥1400. It is **not** the open-sheet corpus MAG was fitted on.
- 2. Give an interval.** A bare percentage invites a comparison it cannot support.

3. **Say the human column is a REALISM target, not a skill target** — and give the reason rather than the assertion: within this corpus the same rate does not separate a sub-1100 player from a 1400–1599 player at a detectable size (-0.054 , 95% CI $[-0.539, +0.403]$, MDE 0.674).
4. **Do not add a Smogon 1630 column to that table.** It has no per-turn rate at any cutoff.
5. **If a strong-player column is wanted, it belongs in a different table about bring and build** — where 1630 and 1760 genuinely answer the question. That table is what this artifact provides.

For reference, the human column recomputed on the current corpus with attribution to the acting side (*clean closed ladder, 171,801 moves over 7,040 logs, 2026-08-06*): failed **2.275%**, immune **2.074%**, super-effective **21.005%**, against §1.3's 2.47 / 1.91 / 21.37 on 1,905 games. Close, and stated so the population and the count travel with the numbers — not as a substitute for re-running `realism_report.js`, which pools rather than attributes.

19g. Filed, not fixed — a real defect found on the way

`data/smogon-priors.json` 's `teammates` **array is polluted on 275 of its 284 species**. Kingambit's holds 32 entries where the source file lists 10, and the extras are `intimidate`, `blaze`, `sitrusberry`, `passhoberry` — the *next* species' Abilities and Items rows. The cause is `engine/smogon_priors.js:160 : grab('Teammates', 'Checks and Counters')` terminates on a section these moveset files **do not contain**, so the regex falls through to `$` and swallows the rest of a 14-chunk window that already spans into the following species block. The same file's `abilities`, `items`, `spreads` and `moves` are unaffected — each has a section that really does follow it.

Blast radius today: zero. Nothing in the repository reads `teammates` out of that file; the only occurrence is the writer. It is latent, not live. Not fixed here because `engine/` is being rewritten in parallel and this division does not patch another agent's file mid-flight.

`provenance.js` **classes this artifact's corpus as open-sheet, and it is closed.** `provenance.js:268` is `/games\.(ots|bo3)\.json1/.test(withDeps) ? 'opensheet' : 'ladder'`, and the generator names both open-sheet stores because it reads their rating summaries. Its **primary** corpus is the clean CLOSED ladder, so drift is judged against the wrong ceiling and the artifact prints *CORPUS DRIFT — declares 8,047 games; 9,177 are clean open-sheet now, 12.3%*. That is the same class as the named exception §5 already records for `winrate-backtest.json`, and it needs the same treatment in `provenance.js`, which is `engine/` and was in flight.

It is left flagged on purpose. `provenance.js` honours a declared `population_ceiling`, and declaring one here would set the ceiling equal to the count and silence this artifact's drift check permanently — which is §5e's `--fix` failure exactly: refitting the world so a check goes green. A false-positive warning that says why is better than a real check switched off.

19h. What this artifact does and does not stamp

`source_digests` is at the **top level**, because `provenance.js:685` reads `j.source_digests` and not `j.provenance.source_digests` — a first run put it in the wrong place and broke the content-digest ratchet in `data/provenance-stamp.json`, which may fall and may never rise. It is closed now; the artifact is one of the **2 of 93** verified by content rather than by mtime.

Only the stable inputs are stamped, deliberately. The generator, `engine/quality.js`, `data/quality-filter.json` and all 32 Smogon monthly dump files — every input that is supposed to be frozen, so a change in one is a real event. The three game stores are **not** stamped and the artifact lists them under `provenance.unstamped_inputs` with the reason: they are append-only, the collector runs hourly, and their digest changes every hour by construction. Stamping them would hang a permanent mismatch on the artifact, which is §5a's "mtime cries wolf" wearing a hash. The instrument for an append-only corpus is the declared count, and that is what `corpus.clean_games` is for.

One more small trap, recorded because it is generic: `run_stamp.sourceDigests()` adds a prose `note` key to the map it returns, and `provenance.js` iterates every key of `source_digests` as a path to re-hash. Left in, it prints *"stamped input note cannot be read to verify"* on every run. The generator deletes it and carries the prose in `source_digests_note` instead.

Living-docs obligations this pass did NOT discharge, because the dispatch scoped the write to two files and forbade `status.js --write`: the CHANGELOG entry and version bump, and whether `SUMMARY.md` or the white paper should carry the §1.3 correction. `docs/ROADMAP.md` §1.3 itself is unedited — the rewrite is specified in 19f and is the router's call to place.

Running the backtest

```
SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown node engine/backtest_winrate.js
```

About 13-18 minutes, one process, on 6,886 clean games (809s and 1,046s on two runs). `MAXG=200` thins it for a smoke run, and the artifact records the n it actually scored, so a thinned run cannot be mistaken for a published one. It writes `data/winrate-backtest.json` and the per-game rows beside it. Every seeded configuration reproduces bit-identically across runs; the unseeded legacy `winProb2` arm moved 0.26 accuracy points between two full runs, which is its run-to-run floor.

It stamps the sha256 of every source the leaf reads. `status.js` re-hashes those and prints `CURRENT` or `PRE-CHANGE`, which is a comparison rather than an mtime inference — a checkout moves an mtime without moving code, and the 2026-08-02 artifact was quoted for two days against an engine that had gained "one mega per side" in between.

Done looks like

- `status.js` prints a leaf calibration line that is fresh, adequately powered, and states the reliability curve rather than a verdict string. **Done 2026-08-04** — and the answer is that the leaf is worse than a coin.
- `provenance.js --strict` exits zero.
- Every gate R1–R4 has an artifact. **Done 2026-08-04** — and R1's turned out to disagree with the prose it replaced. An artifact per gate is the floor, not the goal.
- Every gate artifact says which configuration produced it. **Done 2026-08-04 for R2 and R3** via `engine/run_stamp.js`; R1's dump has its own inline copy of the same shape and should call the module. An artifact that records its build still is not enough on its own: R3 records its build and **not its control**, and a divergence rate without the self-disagreement floor beside it is a headline, not a result.
- `REFIT OWED` is either clear or has a dated reason next to it.